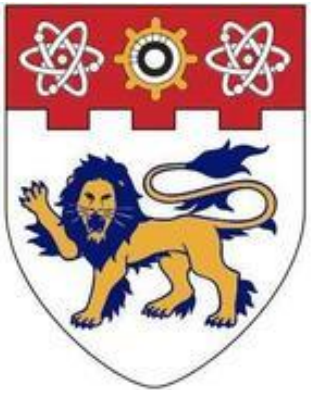


NANYANG TECHNOLOGICAL UNIVERSITY

CCDS25-0821: Gamified AI Companion for Cognitive Health Monitoring in Seniors

Submitted by:

Du Camille Abigail De Jesus

U2222356E

Supervisor: Prof Miao Chun Yan

Submitted in Partial Fulfilment of the Requirements for the Degree of Bachelor of Computer Engineering of the Nanyang Technological University

School of Computing and Data Science

Mar 2026

NANYANG TECHNOLOGICAL UNIVERSITY

Abstract

Mild Cognitive Impairment (MCI) affects an estimated 15 to 20 percent of adults over the age of 65 and significantly increases the risk of progressing to dementia. Early intervention through regular cognitive exercise has been shown to slow this progression, yet existing mobile health (mHealth) applications often suffer from poor long-term adherence because they deliver one-size-fits-all in-game experiences and send reminder notifications at fixed times that do not align with users' natural device engagement patterns.

This report presents CogniFarm, a Unity-based Android mobile application designed to support cognitive health monitoring in older adults through two distinct cognitive mini-games: the Harvesting Memory Game, which exercises sequential memory recall, and the Planting Seeds Game, which trains visual pattern recognition through card matching. Both games incorporate an adaptive difficulty system driven by Q-Learning, which dynamically adjusts challenge levels based on each player's real-time performance metrics to maintain the optimal cognitive challenge state described by Flow Theory.

Beyond in-game adaptation, CogniFarm introduces a personalised push-notification system with two adaptive components. On-device, Thompson Sampling (ReminderAI.cs) maintains Beta(α, β) posteriors over a 192-state context space to decide whether and what type of reminder to send (do not send, gentle, or urgent). Separately, a Firebase Cloud Function implements a 24-arm Thompson Sampling bandit over hours of the day, learning each user's optimal notification hour from proximal engagement signals. Both components are active for the RL group while the rule-based group receives fixed 10am notifications with rule-based content.

The system is evaluated through two complementary experiments. Experiment 1 is part of the benchmarking phase to simulate Thompson Sampling against six alternative policies (Rule-Based, Q-Learning, SARSA, LinUCB, Epsilon-Greedy, Random) using the HeartSteps V1 mHealth dataset and Oralytics dataset as calibration sources for

realistic hourly engagement distributions, with more emphasis placed on the HeartSteps dataset. Experiment 2 assesses real-world adherence in a one-week user study with participants randomly assigned to either a rule-based notification group or the Thompson Sampling RL notification group. Session data including accuracy, response time, difficulty level, notification acceptance, and cognitive trend slope are stored in Firebase Firestore for analysis.

Results from Experiment 1 indicate that Thompson Sampling achieves a mean engagement rate approximately 12 to 18 percentage points higher than the fixed rule-based baseline after 90 simulated days, with posterior evolution analysis confirming convergence toward each user's optimal engagement hour. Experiment 2, a one-week pilot study with N=10 participants, provides preliminary evidence of improved in-game cognitive performance for the RL group relative to the rule-based group (accuracy: d=0.36; peak difficulty reached: d=0.58), though results did not reach statistical significance given the small sample size. Full notification timing results from the corrected Thompson Sampling deployment are pending and will be presented at the final project presentation.

The primary contributions of this project are: (1) a practical multi-game mHealth platform for cognitive training on Android with persistent Q-Learning difficulty adaptation; (2) a per-user Thompson Sampling notification system combining on-device reminder policy learning (ReminderAI.cs) and a serverless Cloud Function for personalised hour selection; (3) a reproducible simulation methodology for validating mHealth RL policies calibrated against real-world engagement data; and (4) an integrated cognitive trend detection module using linear regression over session history to flag potential cognitive decline for professional follow-up.

Keywords: cognitive health monitoring, mild cognitive impairment, mobile health, reinforcement learning, Thompson Sampling, Q-Learning, adaptive difficulty, push notifications, gamification, older adults, Unity, Firebase.

Acknowledgements

I would like to thank my supervisor, Prof Miao Chun Yan, for her support during this project. I am especially grateful to Dr Zeng Zhiwei, who provided hands-on guidance, constructive criticism, and consistent encouragement throughout the development of this project.

I am grateful to the participants who volunteered their time for the user study. This work was completed using Unity 6000.3.2f1, Firebase Unity SDK, and Python 3.11. The HeartSteps V1 dataset (Klasnja et al., 2019) was used under its open-access licence for simulation calibration.

Contents

Acknowledgements 4
List of Figures 7
List of Tables 7
List of Algorithms 8
Chapter 1: Introduction 9
1.1 Background and Motivation 9
1.2 Problem Statement 11
1.3 Research Objectives 11
1.4 Scope and Boundaries 12
1.5 Summary of Contributions 13
1.6 Report Organisation 14
Chapter 2: Literature Review 14
2.1 Cognitive Ageing and Mild Cognitive Impairment 14
2.2 Mobile Health Interventions for Cognitive Training 16
2.3 Gamification in Healthcare Applications 18
2.4 Flow Theory and Adaptive Difficulty 19
2.5 Reinforcement Learning Fundamentals 20
2.6 Contextual Bandits and Thompson Sampling 22
2.7 Push Notification Optimisation in mHealth 25
2.8 Evaluation of mHealth RL Systems 25
2.9 Summary and Identified Research Gaps 26
Chapter 3: System Design and Architecture 27
3.1 System Overview 27
3.2 High-Level Architecture 27
3.3 Mini-Game Design 29
3.4 Adaptive Difficulty Module 32
3.5 Notification Timing Module 38
3.6 Data Layer 40
3.7 Design Decisions and Rationale 44
Chapter 4: Methodology 45
4.1 Research Design 45

4.2 Q-Learning for Adaptive Difficulty 45
4.3 Thompson Sampling for Notification Timing 48
4.4 Cognitive Trend Detection 49
4.5 User Study Design 51
4.6 Simulation Methodology for Algorithm Validation 53
Chapter 5: Implementation 54
5.1 Development Environment 54
5.2 Unity Application Layer 55
5.3 Firebase Backend 58
5.4 Firebase Cloud Function for Notification Timing 58
5.5 Android Build Pipeline 60
5.6 Python Simulation Framework 61
Chapter 6: Evaluation and Results 62
6.1 Experiment 1: Algorithm Comparison 62
6.2 Experiment 2: User Study 67
6.3 Statistical Analysis 72
6.4 Qualitative Observations 72
Chapter 7: Discussion 72
7.1 Interpretation of Findings 72
7.2 Limitations 73
7.3 Ethical Considerations 74
7.4 Future Work 75
Chapter 8: Conclusion 76
References 79
Appendices 87
Appendix A: Q-Table Initialisation Pseudocode 87
Appendix B: Thompson Sampling Cloud Function (functions/index.js) 88
Appendix C: Enlarged Figure 1 - System Architecture Diagram 94
Appendix D: Enlarged Figure 8 - Cloud Function Scheduling Flowchart 95

List of Figures

- Figure 1: System architecture diagram of CogniFarm
- Figure 2: Screenshot of Harvesting Memory Game (sequence recall phase)
- Figure 3: Screenshot of the Planting Seeds Game (card grid)
- Figure 4: Screenshot of the Farm Hub scene
- Figure 5: Q-Learning state space visualization
- Figure 6: Q-Learning update cycle diagram for adaptive difficulty adaptation, shared across both mini-games (HarvestingMemoryGame and PlantingSeedsGame)
- Figure 7: Simulated Beta distribution evolution for a single user over 30 days
- Figure 8: Cloud Function hourly scheduling flowchart
- Figure 9: Firebase Firestore document structure screenshot
- Figure 10: Learning curves for all seven algorithms on HeartSteps V1 and Oralytics
- Figure 11: Final acceptance rate (last 50 training episodes) across HeartSteps V1 and Oralytics for all seven algorithms
- Figure 12: Average days active per 30-day episode across HeartSteps V1 and Oralytics for all seven algorithms
- Figure 13: Accuracy and peak difficulty by session for both groups
- Figure 14: Reminder Engagement Classifications
- Figure 15: Session Adherence and Accuracy Progression by AI Mode

---

List of Tables

- Table 1: Comparison of related mHealth cognitive training applications
- Table 2: Harvesting Game difficulty levels and associated parameters
- Table 3: Planting Seeds Game difficulty levels and parameters
- Table 4: Q-Table initial values by performance level
- Table 5: Q-Learning state-action space
- Table 6: Firestore data fields per session document
- Table 7: User study group allocation and APK build configuration

- Table 8: Thompson Sampling parameterisation
- Table 9: Three-policy comparison
- Table 10: Algorithm configurations for simulation-based comparison (Experiment 1)
- Table 11: Experiment 1 final engagement rates
- Table 12: Normalised performance summary (7-day projected, equal weight per participant)

List of Algorithms

- Algorithm 1: Contextual Thompson Sampling for Reminder Policy
- Algorithm 2: Reward Function
- Algorithm 3: Cloud Function Dispatch Logic
- Algorithm 4: Q-Learning In-Game Difficulty Adaptation

Chapter 1: Introduction

1.1 Background and Motivation

The world is ageing at a rate that healthcare systems are struggling to absorb. By 2030, one in six people globally will be aged 60 or over, and by 2050 the number of people aged 65 and above is projected to reach 1.5 billion (World Health Organization, 2022). Within this demographic shift, cognitive decline represents one of the most consequential and least reversible health trajectories. Dementia currently affects over 55 million people worldwide, with nearly 10 million new cases being diagnosed each year (World Health Organization, 2025). The economic cost of dementia care is estimated to exceed USD 1.3 trillion annually, a figure expected to more than double to USD 2.8 trillion by 2030 (Alzheimer's Disease International, 2023).

A critical window for intervention exists at the stage of Mild Cognitive Impairment (MCI), which is the transitional state between normal age-related cognitive change and dementia. Individuals with MCI exhibit measurable declines in memory, attention, and executive function, yet retain the ability to perform most activities of daily living independently. A major contributing factor to dementia's global prevalence is the issue of delayed or missed diagnosis at this early stage, compounded by the fact that comprehensive MCI assessments are time-intensive and individuals in early stages may not recognise their own impairment (Javed et al., 2023). The annual conversion rate from MCI to Alzheimer's disease is estimated at 10 to 15 percent, compared to just 1 to 2 percent for cognitively healthy individuals (Petersen & Negash, 2008). Crucially, there is growing evidence that structured multidomain interventions including cognitive training can improve cognitive performance in older adults at risk of decline (Ngandu et al., 2015; Lampit et al., 2014).

The widespread adoption of smartphones among older adults offers a unique opportunity to scale cognitive training to a broader population. AARP's 2024 Tech Trends report found that 89 percent of adults over 50 own a smartphone, and this proportion rises every year (Kakulla, 2023). Mobile health (mHealth) applications can be used by individuals in the comfort of their own homes without the logistical barriers of

clinic visits and deliver training in short sessions that fit naturally into daily routines. Many such apps have evolved to use gamified versions of established cognitive tasks, making them more engaging and accessible for at-home use (He et al., 2023). Gamification not only supports clinicians in patient management and provides valuable longitudinal data, but also makes the process of slowing cognitive decline less frustrating and more motivating for patients (Boot et al., 2016).

However, the promise of mHealth for cognitive health has not yet been fully realised. Although doctors recommend regular use of prescribed apps, actual at-home, unsupervised usage is inconsistent (Polzer & Gewald, 2018). As most cognitive health monitoring apps rely on clinical oversight or hybrid models, valid long-term independent use outside clinical settings remains limited (Turunen et al., 2019). Studies consistently find that long-term adherence is the primary barrier, with most cognitive training applications seeing sharp drop-offs in engagement within the first two to four weeks (Baumel et al., 2019).

Two factors drive this drop-off. First, static difficulty does not adapt to the user: tasks that felt challenging become tedious as the user improves, or remain impossibly hard for users with greater impairment, both outcomes leading to disengagement. Second, reminder notifications sent at fixed times are frequently delivered at moments when the user is unavailable, occupied, or simply not in the right frame of mind to engage. A notification received at the wrong time is at best ineffective and at worst actively annoying, reducing the user's overall positive association with the application. Recent research explores the use of AI and Machine Learning methods in gamified cognitive health apps to address these issues, but evidence on their effectiveness in encouraging consistent unsupervised use remains limited, and current approaches often lack sufficient personalisation (Eun et al., 2023; Guzmán et al., 2024).

CogniFarm addresses both of these barriers directly. It applies Q-Learning to maintain each player in their personal optimal difficulty zone across sessions by learning from cumulative performance history, and it applies Thompson Sampling to learn, for each

individual user, the time of day at which they are most likely to open and engage with a notification.

1.2 Problem Statement

Despite a growing body of literature on digital cognitive training and mobile health reinforcement learning, practical deployed systems that combine adaptive in-game difficulty with personalised notification timing in a single cohesive application remain rare, particularly for the older adult demographic. Existing gamified cognitive training applications such as Lumosity and Cognifit incorporate in-game adaptive difficulty, but do not publicly document mechanisms for per-user notification timing personalisation. More broadly, reminder systems in mHealth applications typically deliver notifications at fixed, population-level intervals rather than adapting to individual behavioural patterns (Baumel et al., 2019; Ng et al., 2019).

The specific research questions driving this project are:

Research Question 1. Can Q-Learning effectively personalise in-game difficulty in real-time to maintain older adult players in the optimal cognitive challenge zone as defined by Flow Theory?

Research Question 2. Do per-user Thompson Sampling notification timings produce meaningfully higher engagement rates than a fixed-time rule-based baseline over the course of a pilot study?

Research Question 3. Is simulation using real-world mHealth engagement data (HeartSteps V1) a valid methodology for comparing bandit algorithms prior to deployment, and do simulation results generalise to real user study outcomes?

1.3 Research Objectives

The objectives of this project are:

1. Design and implement a mobile application for cognitive health training on Android, with two mini-games grounded in established neuropsychological tasks:

sequential recall in the Harvesting Memory Game and visual pattern recognition in the Planting Seeds Game.

2. Implement a Q-Learning based adaptive difficulty module within the Harvesting Memory Game and Planting Seeds Game that adjusts challenge level between rounds based on the player's accuracy and response time.
3. Implement a Thompson Sampling contextual bandit on-device (ReminderAI.cs) to learn each user's optimal reminder policy (send/gentle/urgent), and a separate 24-arm Thompson Sampling bandit in a serverless Firebase Cloud Function to learn each user's optimal notification hour from proximal engagement signals.
4. Develop a cognitive trend detection module that analyses longitudinal performance data across sessions using linear regression and raises flags when declining trajectories are detected.
5. Design and conduct a controlled user study with two groups (rule-based notification vs. RL notification) to evaluate the real-world impact of personalised timing on adherence.
6. Validate the Thompson Sampling policy through backtesting on the HeartSteps V1 dataset before deployment and compare that against six alternative algorithms.
7. Integrate all components into a deployable Android application and validate end-to-end functionality on real devices.

1.4 Scope and Boundaries

This project focuses on:

- Android mobile platform (minimum SDK 25, targeting SDK 36).
- Two mini-game modalities: sequential memory recall and card-pair matching.
- Q-Learning for cross-session difficulty adaptation, with a Q-table persisted on-device and an exploration rate that decays across sessions
- Thompson Sampling as a contextual bandit at the notification scheduling layer (not as an in-game agent).
- Firebase Firestore as the backend data store and Firebase Cloud Messaging for push notifications.

- A one-week supervised user study with healthy older adults and younger adult proxies (aged 19 and above) due to ethical and time constraints.

This project does not address:

- Clinical diagnosis of MCI or dementia (CogniFarm is a wellness and self-monitoring tool, not a medical device).
- iOS or web deployment.
- Multimodal sensing (accelerometer, speech, or vision-based inputs).
- Longitudinal studies beyond the FYP study window.

1.5 Summary of Contributions

The main novel contributions of this project are:

1. **CogniFarm platform**: A fully functional Android application combining two memory-focused game modalities derived from standard neuropsychological paradigms (Corsi Block Test for sequential recall; CANTAB-style card matching for visual working memory), persistent Q-Learning difficulty adaptation, and a cognitive trend detection engine, packaged for easy distribution as APK files.

2. **Per-user Thompson Sampling notification scheduler**: A per-user adaptive notification scheduler in which an on-device Thompson Sampling agent maintains separate Beta(α, β) posteriors per user and writes the learned preferred hour to Firestore. A serverless Cloud Function triggered hourly reads this per-user hour and dispatches FCM notifications accordingly.

3. **Simulation validation framework**: A reproducible Python simulation that calibrates the engagement model against two open mHealth MRT datasets (HeartSteps V1 and Oralytics), and compares seven notification policies under identical conditions, providing a pre-deployment validation methodology applicable to any game-based cognitive assessment platform and not limited to the specific games in CogniFarm.

4. **Cognitive trend detection with professional referral trigger**: A linear regression-based trend analyser that monitors accuracy and response time slopes over a configurable session window and surfaces a soft recommendation to consult a healthcare professional when declining trends exceed a heuristically defined threshold, gated on a minimum session count to reduce false positives.

1.6 Report Organisation

Chapter 2 reviews the relevant literature across cognitive ageing, mobile health, gamification, Flow Theory, reinforcement learning, and push notification optimisation. Chapter 3 presents the system design and architecture. Chapter 4 details the methodology for each AI subsystem and the user study. Chapter 5 covers the implementation. Chapter 6 presents evaluation results from both experiments. Chapter 7 discusses the findings, limitations, and future directions. Chapter 8 concludes the report.

Chapter 2: Literature Review

2.1 Cognitive Ageing and Mild Cognitive Impairment

Cognitive ageing is characterised by gradual declines in episodic memory, processing speed, working memory capacity, and executive function, even in the absence of pathology (Salthouse, 2019). These changes begin as early as the third decade of life but become clinically significant for many individuals after the age of 60. The distinction between normal ageing and pathological decline is not always clear, and MCI occupies the uncertain territory between them.

MCI was formally defined by Petersen et al. (1999) as a condition in which a person shows cognitive impairment greater than expected for their age and education, but does not meet the criteria for dementia because daily functioning remains largely intact. The most common subtype, amnestic MCI, is characterised primarily by memory impairment and carries the highest risk of Alzheimer’s conversion (Petersen, 2004). Diagnosis is typically established through neuropsychological testing using instruments such as the

Mini-Mental State Examination (MMSE) or the Montreal Cognitive Assessment (MoCA), often supplemented by structured interviews with caregivers or family members.

The FINGER trial (Ngandu et al., 2015), a landmark randomised controlled trial of 1,260 at-risk adults aged 60 to 77, demonstrated that a multi-domain lifestyle intervention including cognitive training, diet, exercise, and vascular risk monitoring produced a 25% improvement in overall cognitive performance compared to controls after two years. Domain-specific improvements were particularly striking: the intervention group showed approximately 83% greater improvement in executive function, 150% greater improvement in psychomotor speed, and 40% greater improvement in complex memory tasks compared to controls.

The cognitive training component was delivered via a computerised cognitive training programme targeting processing speed, executive function, working memory, and episodic memory, supported by group sessions led by psychologists to assist participants in navigating the tasks (de Jager Loots et al., 2024). However, adherence to the cognitive training component was notably problematic. Compliance was approximately 60% when including participants who completed only the first session. This percentage further dropped to 47% when considering only those who attended over 50% of sessions and completed over 50% of the computer training. The main driver of non-compliance was identified to be computer literacy.

This adherence gap is directly relevant to CogniFarm's design rationale. Even in a highly controlled clinical trial with researcher support, over half of participants failed to sustain meaningful engagement with the computerised cognitive training component. In an unsupervised, at-home deployment context, the adherence challenge is expected to be substantially greater, which motivates the adaptive difficulty, personalised notification timing, and gamified design at the core of this project.

2.2 Mobile Health Interventions for Cognitive Training

mHealth applications for cognitive training have proliferated since the early 2010s, driven by the accessibility of smartphones and the commercial success of products such as Lumosity, BrainHQ, and Cognifit. However, the scientific evidence base for these tools is mixed. A systematic review by Simons et al. (2016) of commercially available brain training applications found that while many produce improvements on trained tasks, evidence for transfer to untrained cognitive domains or real-world functional outcomes remains limited.

More targeted interventions have shown stronger effects. Lampit et al. (2014) conducted a meta-analysis of 52 studies involving computerised cognitive training in older adults and found significant improvements in processing speed (effect size g = 0.31) and verbal memory (d = 0.08), with larger effects for individual sessions over 30 minutes and multi-domain training. It should also be noted that Lampit et al. (2014) found that unsupervised at-home training and sessions more than three times per week were associated with reduced efficacy for cognitive improvement outcomes.

CogniFarm's daily unsupervised design is therefore not expected to optimise cognitive benefit, and this project makes no claims in that regard. The focus of this study is adherence behaviour rather than cognitive efficacy, and the daily session frequency was chosen to maximise the observable adherence signal within the pilot study window.

The shift to smartphone-based delivery introduces both advantages and challenges. Advantages include ubiquitous access, portability, the ability to collect passive sensing data, and the ecological validity of training in the user's own environment. Challenges include the heterogeneity of the older adult population's technological literacy (Barnard et al., 2013), screen size and text legibility constraints, and the higher cognitive demands of navigating unfamiliar software interfaces.

Several papers have documented the adherence problem specifically. Baumel et al. (2019) analysed objective engagement data across mental health applications and

found a 30-day retention rate of just 3.3 percent, with the sharpest drop-off occurring within the first ten days.

Application	Target Population	Game Types	Adaptive Difficulty	Personalised Notifications	RL / Adaptation Method	Platform	Open Source
Lumosity (Hardy et al., 2015)	General adults	Memory, attention, speed, flexibility, problem-solving	Yes	Yes (engagement reminders)	Proprietary (undisclosed)	iOS, Android, Web	No
BrainHQ (Smith et al., 2009)	Adults, seniors, clinical rehab	Processing speed, memory, attention, navigation	Yes	Limited	Staircase adaptive algorithm	iOS, Android, Web	No
CogniFit (Shatil, 2013)	Adults, seniors, clinical populations	Memory, attention, coordination, perception	Yes	Yes	Proprietary adaptive engine	iOS, Android, Web	No
NeuroRacer (Anguera et al., 2013)	Older adults (60–85)	Closed-loop multitasking racing	Yes	No	Closed-loop adaptive (non-RL)	iPad (custom)	No
Peak (Bonnechère et al., 2020)	General adults	Memory, language, mental agility	Yes	Yes	Undisclosed (performance-based)	iOS, Android	No
MindMate (McGoldrick et al., 2019)	Dementia patients, caregivers	Cognitive games, lifestyle tracking	Partial	Yes (caregiver-driven)	Rule-based scheduling	iOS, Android	No
CogniFarm	Seniors (cognitive health monitoring)	Sequence memory, card matching	**Yes**	**Yes**	**Q-Learning (difficulty) + Thompson Sampling contextual bandit (reminder policy)**	Android	No

Table 1: Comparison of related mHealth cognitive training applications

Table 1 summarises several related mHealth cognitive training applications in comparison to CogniFarm.

2.3 Gamification in Healthcare Applications

Gamification refers to the application of game design elements in non-game contexts to increase motivation and engagement (Deterding et al., 2011). In healthcare, gamified interventions have been applied across physical rehabilitation (Burke et al., 2009), diabetes management (Cafazzo et al., 2012), medication adherence (Dayer et al., 2013), and cognitive training (Kueider et al., 2012).

The mechanisms by which gamification drives engagement are grounded in Self-Determination Theory (Brühlmann, 2013) which identifies autonomy, competence, and social relatedness as the three basic psychological needs whose satisfaction leads to intrinsic motivation. Points, levels, and visual feedback address competence by providing concrete signals of progress. In CogniFarm, the farming theme addresses relatedness by situating cognitive tasks within a familiar, low-pressure world. Planting and harvesting crops are activities that carry positive associations for many older adults and provide a coherent narrative reason to return to the application each day. Difficulty adaptation addresses autonomy by ensuring the player is neither overwhelmed nor under-challenged, keeping them in a position where success feels earned rather than trivial or impossible.

For older adults specifically, gamification must account for age-related changes in motor precision, visual acuity, and cognitive load capacity. Falzarano (2023) found that simplified interfaces, large tap targets, high-contrast visuals, and clear written feedback significantly improve usability for adults over 65. CogniFarm’s design incorporates these principles. Buttons are large and clearly coloured, instructions are displayed in large text, and the feedback cycle is immediate and unambiguous.

The farming metaphor was chosen deliberately. Agricultural themes are familiar and culturally neutral, and the concept of planting, tending, and harvesting maps naturally onto the structure of cognitive training: invest effort, see gradual progress, reap reward.

This theme is consistent with findings by Gerling et al. (2012), who recommended metaphors grounded in familiar activities for older adult game design.

2.4 Flow Theory and Adaptive Difficulty

Csikszentmihalyi's Flow Theory (1990) provides the theoretical foundation for CogniFarm's adaptive difficulty system. Flow is described as a state of complete absorption in an activity, characterised by a sense of control, loss of self-consciousness, and intrinsic enjoyment. The central premise relevant to game design is the flow channel. Flow is most likely to occur when the challenge of a task is well matched to the skill of the performer. When challenge exceeds skill, the player experiences anxiety; when skill exceeds challenge, it results in boredom.

For cognitive health applications, maintaining flow is not merely a matter of enjoyment. The therapeutic hypothesis is that cognitive training produces the greatest benefit when it operates at the edge of the player's capability, a concept related to the Vygotskian Zone of Proximal Development (Vygotsky, 1978). If cognitive training is too easy, the brain is not sufficiently challenged to stimulate neuroplasticity. If it is too hard, the player gives up or the experience becomes stressful.

CogniFarm operationalises the flow channel through a reward function centred on a target accuracy range of 60 to 80 percent. This reward function is not itself the adaptive mechanism but rather the training signal that drives the Q-Learning agent's policy which is used to enforce the adaptive mechanism. This will be described in further detail in section 2.5. The optimal zone of 60 to 80 percent yields the maximum positive reward (+1.0). An acceptable range of 40 to 90 percent yields a partial reward (+0.5), representing performance that is within bounds but not ideal. Only accuracy below 40 percent or above 90 percent incurs negative rewards (−0.5 and −0.3 respectively), signalling the anxiety and boredom zones. Response time is incorporated as a secondary signal. Fast responses (+0.3) indicate engaged, confident play, while very slow responses (−0.2) may indicate frustration or cognitive overload.

What distinguishes this from a rule-based adaptive system is what happens next. A rule-based system would directly map these accuracy bands to difficulty actions: if accuracy is low, decrease difficulty; if high, increase it. The Q-Learning agent instead uses the observed reward to update its Q-table via the Bellman equation, incrementally refining its estimate of which action (decrease, maintain, or increase difficulty) produces the highest long-term cumulative reward from each state. Because the state space encodes accuracy, response time, and current difficulty simultaneously, the agent can learn nuanced policies that a fixed rule cannot express: for example, distinguishing between a player who is slow but accurate versus one who is fast but inaccurate, and selecting different difficulty adjustments for each. Over repeated sessions, the Q-table converges toward a personalised policy shaped by each user's individual performance trajectory rather than population-level thresholds.

This approach builds on work by Lopes et al. (2011), who demonstrated that automated difficulty adaptation based on performance metrics significantly improves player retention and perceived enjoyment in games, and on work by Hunicke (2005), whose Dynamic Difficulty Adjustment framework provides the blueprint that CogniFarm's Q-Learning implementation extends from with learned, rather than hard-coded, transition policies.

2.5 Reinforcement Learning Fundamentals

Reinforcement Learning (RL) is the branch of machine learning concerned with training agents to make sequential decisions through interaction with an environment to maximise a cumulative reward signal (Sutton and Barto, 2018). The core formalism is the Markov Decision Process (MDP), defined by a tuple (S, A, P, R, γ) where S is the state space, A the action space, P the transition probability function, R the reward function, and γ a discount factor weighting future rewards against immediate ones.

The Bellman optimality equation defines the value of taking action a in state s as:

$$Q^*(s,a) = R(s,a) + \gamma \, max_{a'} \, Q^*(s',a') \quad (1)$$

Where s' is the resulting state after action a is taken from state s.

Q-Learning (Watkins and Dayan, 1992) is a model-free, off-policy algorithm that approximates Q^* by iteratively updating a table of Q-values using observed transitions:

$$Q(s,a) \leftarrow Q(s,a) + \alpha[r + \gamma \cdot max_{a'} Q(s',a') - Q(s,a)] \quad (2)$$

Where α is the learning rate and r is the observed reward. The algorithm converges to the optimal Q-function under the standard conditions: finite MDP, sufficient exploration, and appropriate learning rate schedule.

For problems where the MDP transition model is not known in advance and the state space is small enough to enumerate, such as in CogniFarm's difficulty adaptation problem, tabular Q-Learning is both practical and theoretically sound. The state space in CogniFarm is a cross product of four performance levels, three response time levels, and five difficulty levels (4 x 3 x 5 = 60 states), which is well within the range where tabular methods are efficient.

The exploration-exploitation tradeoff is managed with an epsilon-greedy policy. With probability epsilon, the agent selects a random action (exploration). Otherwise, it selects the action with the highest Q-value (exploitation).

The exploration rate ε is initialised at $\varepsilon\_0 = 0.3$ and decayed multiplicatively after each completed session t according to:

$$\varepsilon\_(t+1) = max(\varepsilon_{min,} \varepsilon_t \cdot \lambda) \quad (3)$$

Where $\lambda = 0.95$ is the decay factor and $\varepsilon\_min = 0.02$ is the minimum floor, reached after approximately 60 sessions. This schedule ensures that the agent begins with meaningful random exploration necessary to populate player-specific Q-values across the $|S| = 60$ state space, and progressively shifts toward exploiting the learned policy as t increases. Although informed prior Q-values initialise the table with sensible defaults, a non-trivial ε_0 is still required for the agent to discover deviations from those priors that are specific to individual players.

2.6 Contextual Bandits and Thompson Sampling

The push notification problem in CogniFarm has a different structure from the in-game difficulty problem. Notifications are issued once per day, and the binary response of engaged or not engaged is observed the same day. There is no long-horizon sequential dependence between notification decisions across days. The primary uncertainty is not a single unknown parameter but a per-user, per-context question: given what is known about this user right now such as their inactivity level, streak, time of day, and response history, what type of reminder, if any, is most likely to produce engagement? This is precisely the contextual bandit setting (Wu, 2024), where the context observed at decision time partially determines which action is optimal, and the reward is binary engagement.

Thompson Sampling (Thompson, 1933) is a Bayesian approach to the bandit problem that maintains a posterior distribution over the reward probability of each action and selects actions by sampling from these posteriors. For a binary reward with Beta prior, the posterior for each state–action pair (s, a) is maintained as:

$$Beta(\alpha\_(s,a), \beta\_(s,a)) \tag{4}$$

Where $\alpha\_(s,a)$ accumulates successes and $\beta\_(s,a)$ accumulates failures.

The algorithm is:

Algorithm 1: Contextual Thompson Sampling for Reminder Policy

Initialise: For each state $s \in S$ ($|S| = 192$) and action $a \in \{0, 1, 2\}$:

    $\alpha\_\{s, a\} = 1$ (Beta prior: pseudo-successes)

    $\beta\_\{s, a\} = 1$ (Beta prior: pseudo-failures)

Actions:

    0 = do not send

    1 = send gentle reminder

    2 = send urgent reminder

State s = (time_slot, inactivity_level, last_response, streak_level)

For each decision point $t = 1, 2, \ldots$:

    Observe current context s_t

    For each action $a \in \{0, 1, 2\}$:

        Sample $\theta\_\{s\_t, a\} \sim Beta(\alpha\_\{s\_t, a\}, \beta\_\{s\_t, a\})$

    Select $a\_t = argmax\_a \, \theta\_\{s\_t, a\}$

    Execute action a_t; observe reward $r\_t \in \{0, 1\}$

    Update:

        if $r\_t = 1$: $\alpha\_\{s\_t, a\_t\} \mathrel{+}= 1$ (reminder accepted)

        else: $\beta\_\{s\_t, a\_t\} \mathrel{+}= 1$ (reminder rejected or ignored)

The state space encodes four contextual dimensions: time slot (4 levels), days of inactivity (4 levels), last reminder response (4 levels), and streak level (3 levels), yielding $|S| = 4 \times 4 \times 4 \times 3 = 192$ states. This contextual structure distinguishes the approach from a standard multi-armed bandit: the same action may be optimal in one context and suboptimal in another, and the agent learns this distinction independently per state.

Notification hour selection is handled by a separate Cloud Function component that implements Thompson Sampling over 24 hour arms, maintaining per-player Beta posteriors (tsHourAlpha, tsHourBeta). On-device, a simple moving average of the user's

session timestamps is written to Firestore as preferredHour, serving as a warm-start bootstrap only. The Cloud Function samples from each player's Beta posteriors each hour and updates them via a 2-hour proximal engagement window: if the player opens the app within 2 hours of a sent notification, α is incremented; otherwise β is incremented.

The algorithm achieves asymptotically optimal regret of $O(log\ T)$ (Agrawal and Goyal, 2012; Kaufmann et al., 2012), meaning the gap between its cumulative reward and the oracle policy grows only logarithmically with time. Empirically, Thompson Sampling has been shown to outperform UCB (Upper Confidence bound) and Epsilon-Greedy in terms of sample efficiency across a wide range of bandit settings (Chapelle and Li, 2011; Russo et al., 2018).

The key innovation in CogniFarm relative to most deployed mHealth systems is that the bandit is both per-user and contextual. Each user maintains entirely separate α and β parameters for all 192 state-action pairs, persisted locally to reminder_ts_data.json. This means the system can simultaneously learn that User A responds well to urgent reminders after three days of inactivity but ignores gentle ones, while User B responds to gentle reminders at streak boundaries and disengages when sent urgent reminders.

Prior work on per-user notification timing includes Bidargaddi et al. (2018), who reported a 3.9 percent improvement in engagement using a population-level bandit for mental health app reminders, and Klasnja et al. (2019), who used contextual bandits with rich context features such as location, weather and activity to time walking suggestions. CogniFarm's contribution is a per-user contextual Beta-Bernoulli formulation over reminder policy decisions, which is simpler to implement on-device and more privacy-preserving than approaches requiring continuous sensor context.

2.7 Push Notification Optimisation in mHealth

Push notifications have become the primary re-engagement mechanism for mHealth applications, yet their effectiveness is highly variable. Thaler et al. (2013) found that notifications sent at contextually appropriate times were three times more likely to result in users actually opening apps than notifications sent at random times. Trafton et al. (2005) identified “interruptibility” as the key moderating variable, stating that users who are in the middle of another task when a notification arrives are significantly less likely to respond.

Several studies have modelled notification timing as a reinforcement learning problem. Klasnja et al. (2019) applied a micro-randomised trial design within the HeartSteps study to estimate the causal effect of suggestion timing on step count, providing the methodological framework that informs the simulation validation in this project.

2.8 Evaluation of mHealth RL Systems

Evaluating RL systems in mHealth is methodologically challenging. Randomised controlled trials are the gold standard but require large sample sizes and long follow-up periods. Within this FYP’s constraints, a one-week study with a small sample was conducted. The simulation-based evaluation framework adopted in this project follows the approach described by Tewari and Murphy (2017), who argue that simulation calibrated against real mHealth data is a legitimate and necessary pre-deployment validation step, particularly when the target population is difficult to recruit.

Two open-access mHealth datasets are used as simulation ground truth: the HeartSteps V1 dataset (Klasnja et al., 2019), a walking encouragement study with 37 participants over six weeks, and the Oralytics dataset (Trella et al., 2024), a micro-randomised trial on adaptive digital oral health interventions. Both contain timestamped interaction records with binary engagement outcomes, yielding empirical engagement probability curves for simulation calibration. These probability curves serve as the ground truth for the simulation in Experiment 1, allowing a rigorous comparison of algorithm performance under realistic conditions.

The simulation methodology used in this project is explicitly described as simulation rather than real data, clearly distinguished from Experiment 2 which is the actual user study, and framed as a pre-deployment algorithm validation exercise. This is consistent with established practice in mHealth RL research (Liao et al., 2020).

2.9 Summary and Identified Research Gaps

The review of the literature reveals several gaps that CogniFarm addresses:

Gap 1: Few deployed systems combine adaptive in-game difficulty with personalised notification timing in a single application. Most mHealth cognitive training tools address either the engagement problem of adaptive difficulty or the adherence problem of notification timing, but not both simultaneously.

Gap 2: Notification personalisation research typically optimises at the population level, not the per-user level. Per-user bandit approaches that are based on learning individually when and how to intervene are theoretically superior but rarely implemented in practice, partly due to the engineering complexity of maintaining per-user state.

Gap 3: Cognitive trend detection is largely absent from consumer-facing mHealth applications. Apps like Lumosity report aggregate scores but do not provide the user with a clear signal about whether their performance is trending in a direction that warrants clinical attention.

Gap 4: Simulation validation frameworks for mHealth RL are not standardised or reusable. The simulation tool developed in this project is calibrated against the HeartSteps V1 dataset (Klasnja et al., 2019) and the Oralytics dataset (Trella et al., 2024). It compares seven algorithms, addressing this gap and providing a reusable baseline for future projects.

Chapter 3: System Design and Architecture

3.1 System Overview

CogniFarm is a client-server mobile health application. The client is a Unity-based Android application that runs two cognitive mini-games with an AI system to determine each player's unique adapted difficulty levels. The server comprises Firebase Firestore which serves as the real-time database, Firebase Cloud Messaging (FCM) for push notifications, and Firebase Cloud Functions (serverless Node.js functions) for backend logic. All components are hosted on Google Cloud Platform through the Firebase project.

The application is distributed as two distinct APK builds for the user study. One is configured for rule-based notification group where notifications are sent at a fixed 10:00 SGT and each mini-game's difficulty follows rule-based algorithm. The other is for the RL group where on-device Thompson Sampling determines the reminder policy (whether and at what urgency to send), the preferred notification hour is derived from a moving average of session timestamps and stored in Firestore, and each mini-game's difficulty follows the dynamic Q-Learning algorithm. Both builds share the same game code, session data export logic, and Firebase project. The only differences are the `studyGroup` field hardcoded in the `GameDataExporter.cs` Inspector, and the Cloud Function's handling of the `studyGroup` field per player.

3.2 High-Level Architecture

Below is a figure showing the system architecture of CogniFarm. An enlarged version of Figure 1 is available in the appendix.

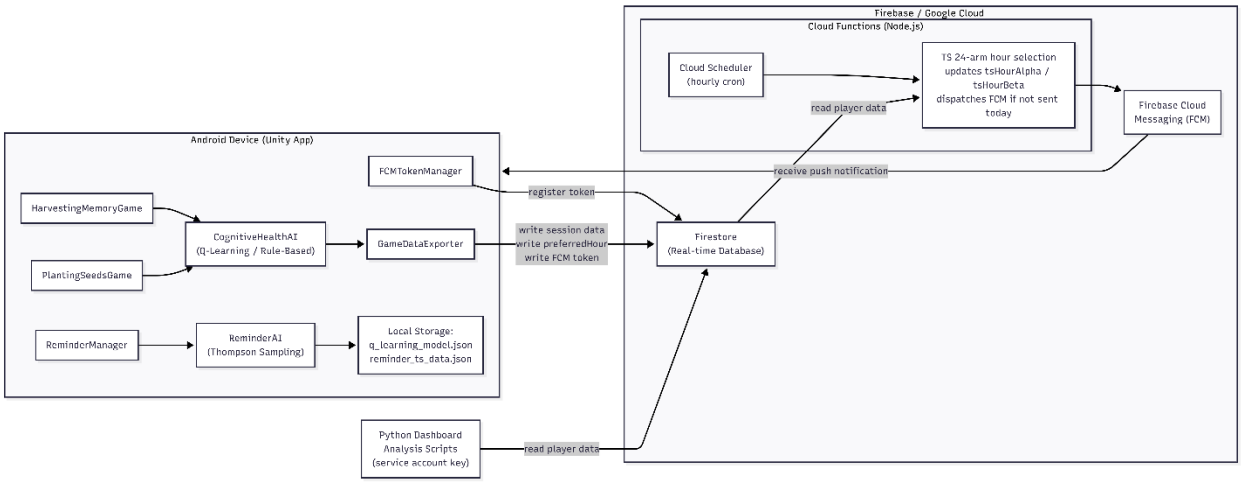


Figure 1: System Architecture Diagram of CogniFarm

The architecture consists of four layers:

Layer 1: Presentation (Unity/C#). The Unity application handles all game logic, UI rendering, user input, and local persistence. The HarvestingMemoryGame.cs and PlantingSeedsGame.cs MonoBehaviours manage game flow. The CognitiveHealthAI.cs MonoBehaviour runs the Q-Learning difficulty adapter and cognitive trend detector. The GameDataExporter.cs MonoBehaviour handles Firestore writes and session data serialisation.

Layer 2: Communication. The FCMTokenManager.cs MonoBehaviour registers the device with FCM on startup and writes the token to Firestore. The ReminderManager.cs MonoBehaviour handles local notification scheduling as a fallback.

Layer 3: Backend Logic (Node.js Cloud Function). The index.js Cloud Function runs on an hourly schedule via Cloud Scheduler. For each player in Firestore, the Cloud Function samples from per-player Beta posteriors (tsHourAlpha, tsHourBeta) across 24 hour arms to select the send hour, dispatching an FCM notification if one has not already been sent today. The preferredHour field written on-device serves as a warm-start bootstrap for initialising these priors. The Cloud Function also implements

Thompson Sampling over 24 hour arms per player, maintaining tsHourAlpha and tsHourBeta posteriors and updating them via a 2-hour proximal engagement window.

Layer 4: Data and Analytics. Firestore stores all player documents, session data, and notification hour posteriors. cognifarm_analysis.py connects to Firestore via a service account key to download session data for analysis. cognifarm_dashboard.py is a Streamlit dashboard that reads exported local data files and provides visualisations for the researcher.

3.3 Mini-Game Design

3.3.1 Harvesting Memory Game

The Harvesting Memory Game is a sequence recall task in which the player must observe a highlighted sequence of coloured crop tiles on a grid sized either 3×3 or 4×4 depending on the round difficulty and reproduce the sequence from memory. The game draws on established neuropsychological paradigms including the Corsi Block Test (Corsi, 1972) for spatial span and Digit Span tasks from the Wechsler Adult Intelligence Scale.

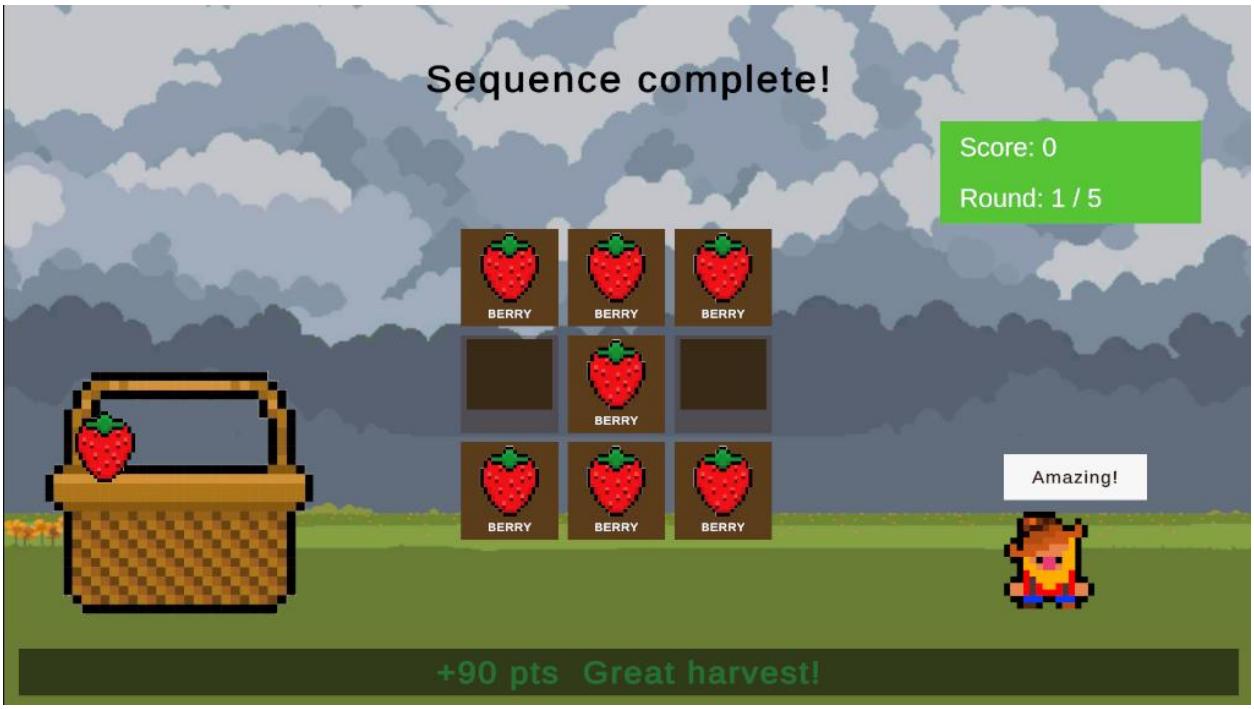


Figure 2: Screenshot of Harvesting Memory Game (sequence recall phase)

The game proceeds in rounds. At the start of each round, a sequence of grid positions is highlighted one at a time in a predetermined order. After a brief memorisation period, the tiles return to their original colours and the player must tap them in the correct order. Sequence length, highlight duration, inter-stimulus delay, and grid size all vary with the current difficulty level.

There are five difficulty levels defined as follows:

Level	Name	Sequence Length	Highlight Duration (s)	ISI (s)	Memorisation (s)	Grid Size
1	Very Easy	2	1.0	0.6	4.0	3×3
2	Easy	3	1.0	0.6	3.0	3×3
3	Medium	4	1.0	0.6	2.5	3×3
4	Hard	5	1.0	0.6	2.0	4×4
5	Very Hard	6	1.0	0.6	1.5	4×4

Table 2: Harvesting Game difficulty levels and associated parameters

Scoring rewards both sequence length and difficulty level:

$$round\_score = 50 + (sequence\_length * 10) + (difficulty * 20) \quad (5)$$

3.3.2 Planting Seeds Game

The Planting Seeds Game is a card-matching concentration game in which the player must flip pairs of cards to find matching seed and crop pairs. This game exercises visual working memory and pattern recognition, drawing on the Well-Matched Pairs paradigm used in the Cambridge Neuropsychological Test Automated Battery (CANTAB) (Robbins et al., 1994).

Figure 3: Screenshot of the Planting Seeds Game (card grid)

Cards are arranged in a grid whose size scales with difficulty. Two cards form a pair if they share the same pairId. The player flips cards two at a time and a match is awarded if both cards share the same pair ID. The number of pairs, preview time, and card face-down delay all scale with difficulty. The number of pairs increases while the flip-back delay decreases, requiring the player to retain more card positions for a shorter window of time. This follows the same five-level scale as the Harvesting Game.

Level	Name	Pairs	Total Cards	Preview Time (s)	Flip-back Delay (s)
1	Very Easy	3	6	4.0	1.5
2	Easy	4	8	5.0	1.2
3	Medium	5	10	6.0	1.0
4	Hard	6	12	7.0	0.8
5	Very Hard	8	16	8.0	0.6

Table 3: Planting Seeds Game difficulty levels and parameters

3.3.3 Farm Hub

The Farm Hub is the application's navigation hub scene. From here, the player can select either mini-game, view their performance history, and access the AI companion summary screen. The farming visual theme of green fields, crop icons, and pastoral colours provides the connecting narrative that ties both games into a coherent experience.

Figure 4: Screenshot of Farm Hub scene

3.4 Adaptive Difficulty Module

The adaptive difficulty module is implemented in `CognitiveHealthAI.cs` and `HarvestingMemoryGame.cs`. The module runs at the end of each round in the Harvesting Game and at the end of each round in the Planting Seeds Game.

State Space

The Q-Learning agent perceives the game state as a three-dimensional vector:

- **Performance Level (0-3):** Poor (accuracy < 40%), Fair (40-60%), Good (60-80%), Excellent (>80%)
- **Response Time Level (0-2):** Slow (> 10s), Normal (5-10s), Fast (< 5s)
- **Current Difficulty (0-4, indexed from 1-5)**

This gives a total of $4 \times 3 \times 5 = 60$ states. Below is the state index formula:

$$performanceLevel \times 15 + responseTimeLevel \times 5 + (difficulty - 1) \quad (6)$$

Action Space

The agent chooses from three actions:

Action 0: Decrease difficulty by one level (minimum 1)

Action 1: Maintain current difficulty

Action 2: Increase difficulty by one level (maximum 5)

Reward Function

The reward is calculated based on the player's recent accuracy and response time to measure how close they are to the optimal flow state:

Algorithm 2: Reward Function

$reward = 0$

```
if 60% <= accuracy <= 80%:
    reward += 1.0      # Optimal flow zone
elif 40% <= accuracy <= 90%:
    reward += 0.5      # Acceptable range
elif accuracy < 40%:
    reward -= 0.5      # Too hard
elif accuracy > 90%:
    reward -= 0.3      # Too easy

if response_time < 8s:
    reward += 0.3      # Good engagement
elif response_time > 15s:
    reward -= 0.2      # Possible frustration

if round_was_successful:
    reward += 0.2      # Reinforcement for success
```

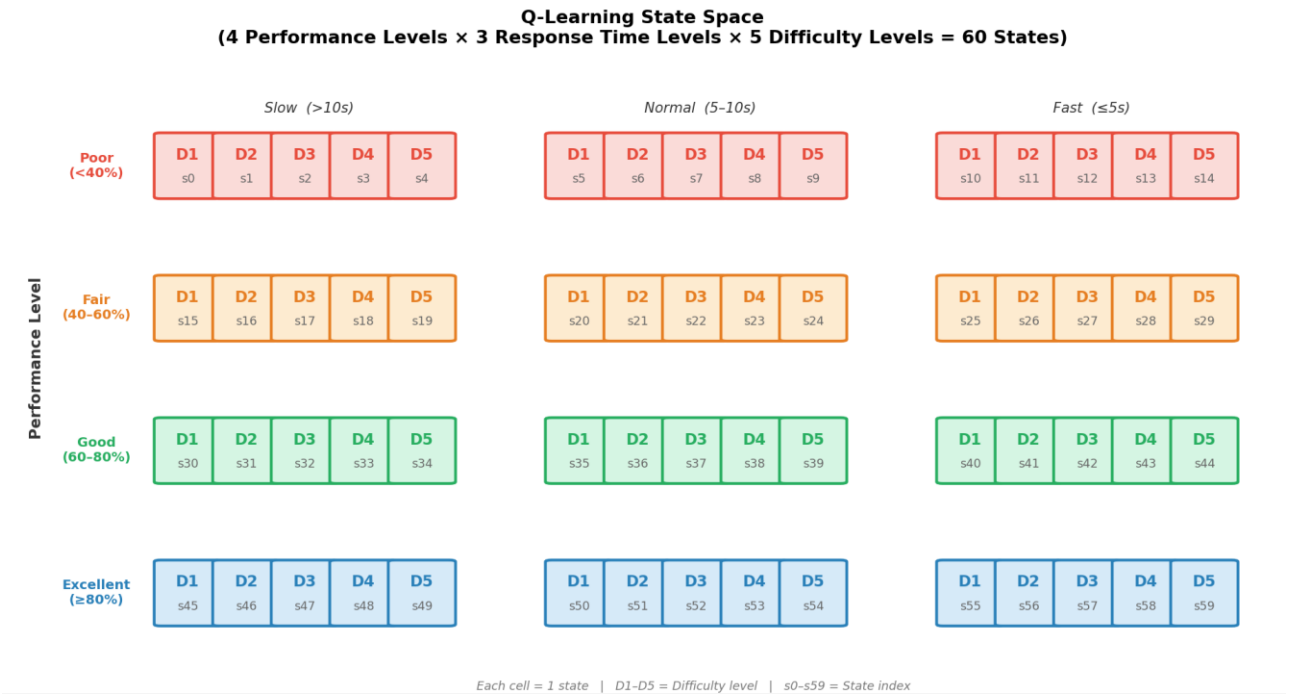


Figure 5: Q-Learning state space visualization

Q-Table Initialisation

Rather than initialising the Q-table to zero which would require extensive exploration before producing useful behaviour, CogniFarm uses informed prior values that encode sensible behaviour from session one. Poor performance biases the agent toward decreasing difficulty; Excellent performance biases it toward increasing difficulty; Good performance (the target zone) biases it toward maintaining. Small uniform noise (±0.05) is added to break ties during exploration.

Performance Level	Decrease Q	Maintain Q	Increase Q
Poor (0)	+1.0	-0.5	-1.0
Fair (1)	+0.3	+0.5	-0.3
Good (2)	-0.3	+1.0	+0.3
Excellent (3)	-1.0	-0.3	+1.0

Table 4: Q-Table initial values by performance level

State	Perf Level	RT Level	Difficulty	Q(Decrease)	Q(Maintain)	Q(Increase)
0	Poor (<40%)	Slow (>10s)	1	+1.0	−0.5	−1.0
1	Poor (<40%)	Slow (>10s)	2	+1.0	−0.5	−1.0
2	Poor (<40%)	Slow (>10s)	3	+1.0	−0.5	−1.0
3	Poor (<40%)	Slow (>10s)	4	+1.0	−0.5	−1.0
4	Poor (<40%)	Slow (>10s)	5	+1.0	−0.5	−1.0
5	Poor (<40%)	Normal (5–10s)	1	+1.0	−0.5	−1.0
6	Poor (<40%)	Normal (5–10s)	2	+1.0	−0.5	−1.0
7	Poor (<40%)	Normal (5–10s)	3	+1.0	−0.5	−1.0
8	Poor (<40%)	Normal (5–10s)	4	+1.0	−0.5	−1.0
9	Poor (<40%)	Normal (5–10s)	5	+1.0	−0.5	−1.0
10	Poor (<40%)	Fast (≤5s)	1	+1.0	−0.5	−1.0
11	Poor (<40%)	Fast (≤5s)	2	+1.0	−0.5	−1.0
12	Poor (<40%)	Fast (≤5s)	3	+1.0	−0.5	−1.0
13	Poor (<40%)	Fast (≤5s)	4	+1.0	−0.5	−1.0
14	Poor (<40%)	Fast (≤5s)	5	+1.0	−0.5	−1.0
15	Fair (40–60%)	Slow (>10s)	1	+0.3	+0.5	−0.3
16	Fair (40–60%)	Slow (>10s)	2	+0.3	+0.5	−0.3
17	Fair (40–60%)	Slow (>10s)	3	+0.3	+0.5	−0.3
18	Fair (40–60%)	Slow (>10s)	4	+0.3	+0.5	−0.3
19	Fair (40–60%)	Slow (>10s)	5	+0.3	+0.5	−0.3
20	Fair (40–60%)	Normal (5–10s)	1	+0.3	+0.5	−0.3
21	Fair (40–60%)	Normal (5–10s)	2	+0.3	+0.5	−0.3
22	Fair (40–60%)	Normal (5–10s)	3	+0.3	+0.5	−0.3
23	Fair (40–60%)	Normal (5–10s)	4	+0.3	+0.5	−0.3
24	Fair (40–60%)	Normal (5–10s)	5	+0.3	+0.5	−0.3
25	Fair (40–60%)	Fast (≤5s)	1	+0.3	+0.5	−0.3
26	Fair (40–60%)	Fast (≤5s)	2	+0.3	+0.5	−0.3
27	Fair (40–60%)	Fast (≤5s)	3	+0.3	+0.5	−0.3
28	Fair (40–60%)	Fast (≤5s)	4	+0.3	+0.5	−0.3
29	Fair (40–60%)	Fast (≤5s)	5	+0.3	+0.5	−0.3
30	Good (60–80%)	Slow (>10s)	1	−0.3	+1.0	+0.3
31	Good (60–80%)	Slow (>10s)	2	−0.3	+1.0	+0.3
32	Good (60–80%)	Slow (>10s)	3	−0.3	+1.0	+0.3
33	Good (60–80%)	Slow (>10s)	4	−0.3	+1.0	+0.3
34	Good (60–80%)	Slow (>10s)	5	−0.3	+1.0	+0.3
35	Good (60–80%)	Normal (5–10s)	1	−0.3	+1.0	+0.3
36	Good (60–80%)	Normal (5–10s)	2	−0.3	+1.0	+0.3
37	Good (60–80%)	Normal (5–10s)	3	−0.3	+1.0	+0.3
38	Good (60–80%)	Normal (5–10s)	4	−0.3	+1.0	+0.3
39	Good (60–80%)	Normal (5–10s)	5	−0.3	+1.0	+0.3
40	Good (60–80%)	Fast (≤5s)	1	−0.3	+1.0	+0.3
41	Good (60–80%)	Fast (≤5s)	2	−0.3	+1.0	+0.3
42	Good (60–80%)	Fast (≤5s)	3	−0.3	+1.0	+0.3
43	Good (60–80%)	Fast (≤5s)	4	−0.3	+1.0	+0.3
44	Good (60–80%)	Fast (≤5s)	5	−0.3	+1.0	+0.3
45	Excellent (≥80%)	Slow (>10s)	1	−1.0	−0.3	+1.0
46	Excellent (≥80%)	Slow (>10s)	2	−1.0	−0.3	+1.0
47	Excellent (≥80%)	Slow (>10s)	3	−1.0	−0.3	+1.0
48	Excellent (≥80%)	Slow (>10s)	4	−1.0	−0.3	+1.0
49	Excellent (≥80%)	Slow (>10s)	5	−1.0	−0.3	+1.0
50	Excellent (≥80%)	Normal (5–10s)	1	−1.0	−0.3	+1.0
51	Excellent (≥80%)	Normal (5–10s)	2	−1.0	−0.3	+1.0
52	Excellent (≥80%)	Normal (5–10s)	3	−1.0	−0.3	+1.0
53	Excellent (≥80%)	Normal (5–10s)	4	−1.0	−0.3	+1.0
54	Excellent (≥80%)	Normal (5–10s)	5	−1.0	−0.3	+1.0
55	Excellent (≥80%)	Fast (≤5s)	1	−1.0	−0.3	+1.0
56	Excellent (≥80%)	Fast (≤5s)	2	−1.0	−0.3	+1.0
57	Excellent (≥80%)	Fast (≤5s)	3	−1.0	−0.3	+1.0
58	Excellent (≥80%)	Fast (≤5s)	4	−1.0	−0.3	+1.0
59	Excellent (≥80%)	Fast (≤5s)	5	−1.0	−0.3	+1.0

Table 5: Q-Learning state-action space

Table 5 shows the initial Q-values prior to any gameplay. Priors are seeded by performance level only. Response time and current difficulty level do not influence initialisation. Q-values diverge per-state from session 1 onwards as the Bellman update applies to each of the 60 states independently based on the player's actual trajectory.

Q-Table Persistence

The Q-table is serialised to JSON and saved to `Application.persistentDataPath` at the end of every session via `DecayExploration()`. On subsequent sessions, the saved table is loaded, preserving learned behaviour across sessions. A version tag prevents loading incompatible tables after code changes.

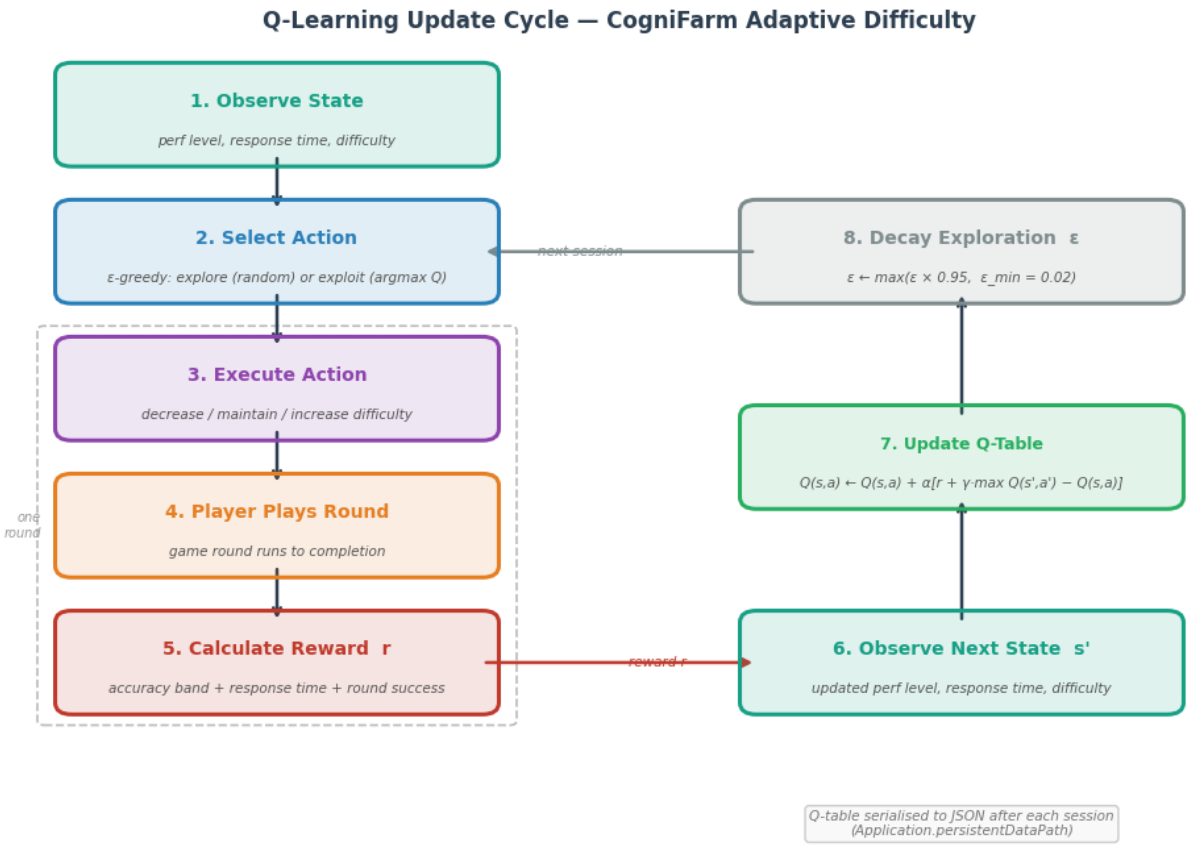


Figure 6: Q-Learning update cycle diagram for adaptive difficulty adaptation, shared across both mini-games (HarvestingMemoryGame and PlantingSeedsGame)

3.5 Notification Timing Module

Overview

For each user, the system maintains a Beta(α, β) posterior for every combination of context state and action. The state space encodes four contextual dimensions: time of day (4 slots), days since last session (4 levels), the player's last notification response (4 types), and current streak status (3 levels). This yields 192 states. Across each state, three actions are tracked: send no reminder (0), send a gentle reminder (1), or send an urgent reminder (2). This gives a total of 576 state-action posteriors per user, each initialised at Beta(1, 1). These posteriors are persisted locally on-device in reminder_ts_data.json and updated each time an engagement outcome is observed.

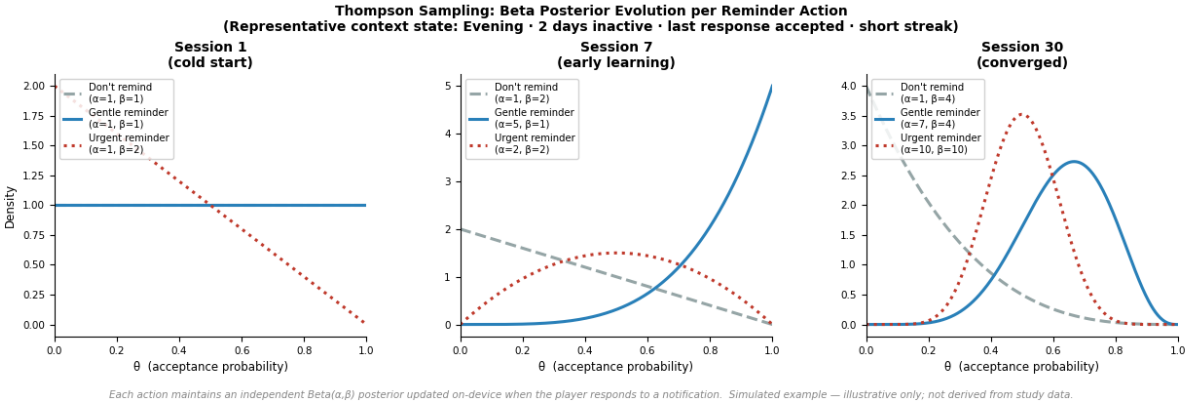


Figure 7: Simulated Beta distribution evolution for a single user over 30 days

Figure 7 illustrates how the Thompson Sampling agent learns the optimal reminder action for a given context state over time. Each action of Don't remind, Gentle reminder, and Urgent reminder, starts with an uninformative flat prior (Beta(1,1)), meaning the agent has no initial preference. As sessions pass and engagement outcomes are recorded on-device, the posterior for the most effective action gradually sharpens into a peaked distribution concentrated around its true acceptance rate, while less effective actions remain flat or shift toward lower probability. In this simulated example, within a representative context state (Evening, 2 days inactive, last response accepted, short

streak), the Gentle reminder action emerges as dominant by Session 30 (α=7, β=4) while Urgent reminder and Don't remind remain uncertain or low. Note that this is simulated data for illustration purposes only. In real deployment, convergence speed and the dominant action will vary per player and per context state across all 192 states.

Cloud Function Logic

The Cloud Function (sendDailyReminders) is triggered hourly by Cloud Scheduler (cron: 0 * * * *, timezone: Asia/Singapore) and acts as a pure dispatcher. On-device Thompson Sampling determines the reminder policy (whether and what to send). Hour selection runs server-side in the Cloud Function via 24-arm Thompson Sampling. The Cloud Function steps are as follows:

Algorithm 3: Cloud Function Dispatch Logic.

1. Read the current SGT hour h_now.
2. Query all player documents from Firestore.
3. For each player:
    - If lastReminderDate equals today's date, skip.
    - If no FCM token is stored, skip.
    - For each hour $arm\ h \in \{0 \dots 23\}$, $sample\ \theta\_h \sim Beta(tsHourAlpha[h], tsHourBeta[h])$.
    - Select target hour $h^* = argmax\ \theta\_h$.
    - If $h_{now} \neq h^*$, skip.
    - Evaluate reminder content using getReminderDecision (rule-based for Rule_Based Groups; TS content for RL group when available).
    - If the decision is to send, dispatch FCM via Firebase Admin SDK, mark lastReminderDate, log to reminders subcollection, and update $tsHourAlpha[h^*]$ or $tsHourBeta[h^*]$ based on proximal engagement window.

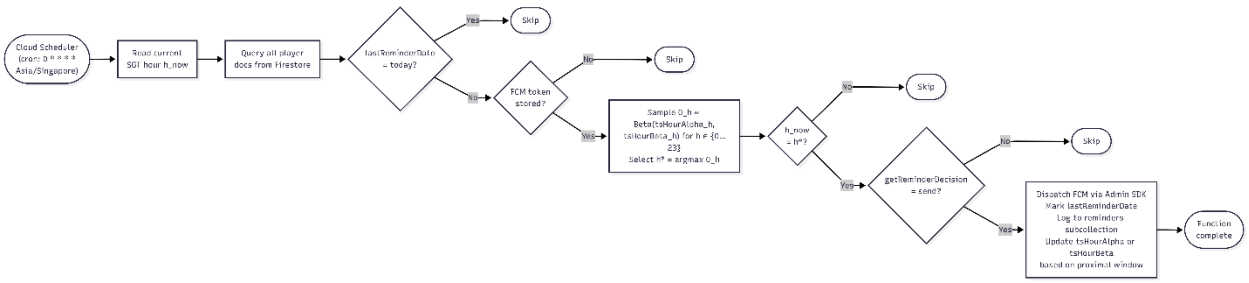


Figure 8: Cloud Function hourly scheduling flowchart

An enlarged version of Figure 8 is available in the appendix.

Engagement Signal

The Thompson Sampling α/β update happens on-device. Because FCM provides no tap or open callback to Unity or Firestore, engagement cannot be directly observed. Instead, CogniFarm adopts a proximal outcome design consistent with HeartSteps (Klasnja et al., 2019): if the player's `lastSessionTimestamp` in Firestore falls within two hours of the notification's `sentAt` timestamp, the notification is treated as accepted and α is incremented for the chosen state-action pair. If no session is recorded within that window, the notification is treated as ignored and β is incremented instead. This heuristic is evaluated on-device the next time the app is opened, by comparing the stored `sentAt` from the most recent reminder log entry against the current `lastSessionTimestamp`.

3.6 Data Layer

All game session data is written to Firebase Firestore by GameDataExporter.cs at the end of each game session. The Firestore collection structure is:

```
players/{playerId}/
    playerId: string
```

```
    studyGroup: string              ("rule_based" | "rl")
    preferredHour: number           (moving average of session timestamps)
    reminderAIMode: string          ("ThompsonSampling" | "RuleBased")
    remindersTotal: number
    remindersAccepted: number
    remindersRejected: number
    reminderAcceptanceRate: float
    firstPlayDate: string
    lastPlayDate: string
    totalSessionsRecorded: number
    completedSessions: number
    abandonedSessions: number
    currentStreak: number
    longestStreak: number
    participantName: string         (researcher mapping only)
    age: number
    lastUpdated: timestamp
    tsHourAlpha: array[24]   (Beta α posteriors for hour selection, Cloud Function)
    tsHourBeta:  array[24]   (Beta β posteriors for hour selection, Cloud Function)

players/{playerId}/sessions/{sessionId}/
    sessionId: string
    playerId: string
    timestamp: string
    date: string
    time: string
    gameType: string                ("Harvesting" | "Planting")
    aiMode: string                  ("Q-Learning" | "Rule-Based")
    sessionStatus: string           ("Completed" | "Abandoned")
    totalRounds: number
    expectedRounds: number
    successfulRounds: number
    accuracy: float
    avgResponseTime: float
    startDifficulty: number
    endDifficulty: number
    peakDifficulty: number
```

```
difficultyChanges: number
finalScore: number
cognitiveAssessment: string
uploadTime: timestamp
```

Field	Type	Description
sessionId	string	Auto-generated 8-character unique session identifier
playerId	string	Player identifier (e.g. P_1C842436)
timestamp	string	Session start datetime (YYYY-MM-DD HH:mm:ss)
date	string	Session date (YYYY-MM-DD)
time	string	Session start time (HH:mm:ss)
gameType	string	Mini-game played ("Harvesting" or "Planting")
aiMode	string	Difficulty AI mode ("Q-Learning" or "Rule-Based")
sessionStatus	string	Outcome ("Completed" or "Abandoned")
totalRounds	number	Number of rounds played in session
expectedRounds	number	Target number of rounds for the session
successfulRounds	number	Rounds answered correctly
accuracy	float	Session accuracy as percentage (0–100)
avgResponseTime	float	Mean response time per round in seconds
startDifficulty	number	Difficulty level at session start (1–5)
endDifficulty	number	Difficulty level at session end (1–5)
peakDifficulty	number	Highest difficulty reached during session (1–5)
difficultyChanges	number	Number of AI-triggered difficulty adjustments
finalScore	number	Total score accumulated in session
cognitiveAssessment	string	AI-generated trend summary for result screen
uploadTime	timestamp	Firestore server timestamp of upload

Table 6: Firestore data fields per session document

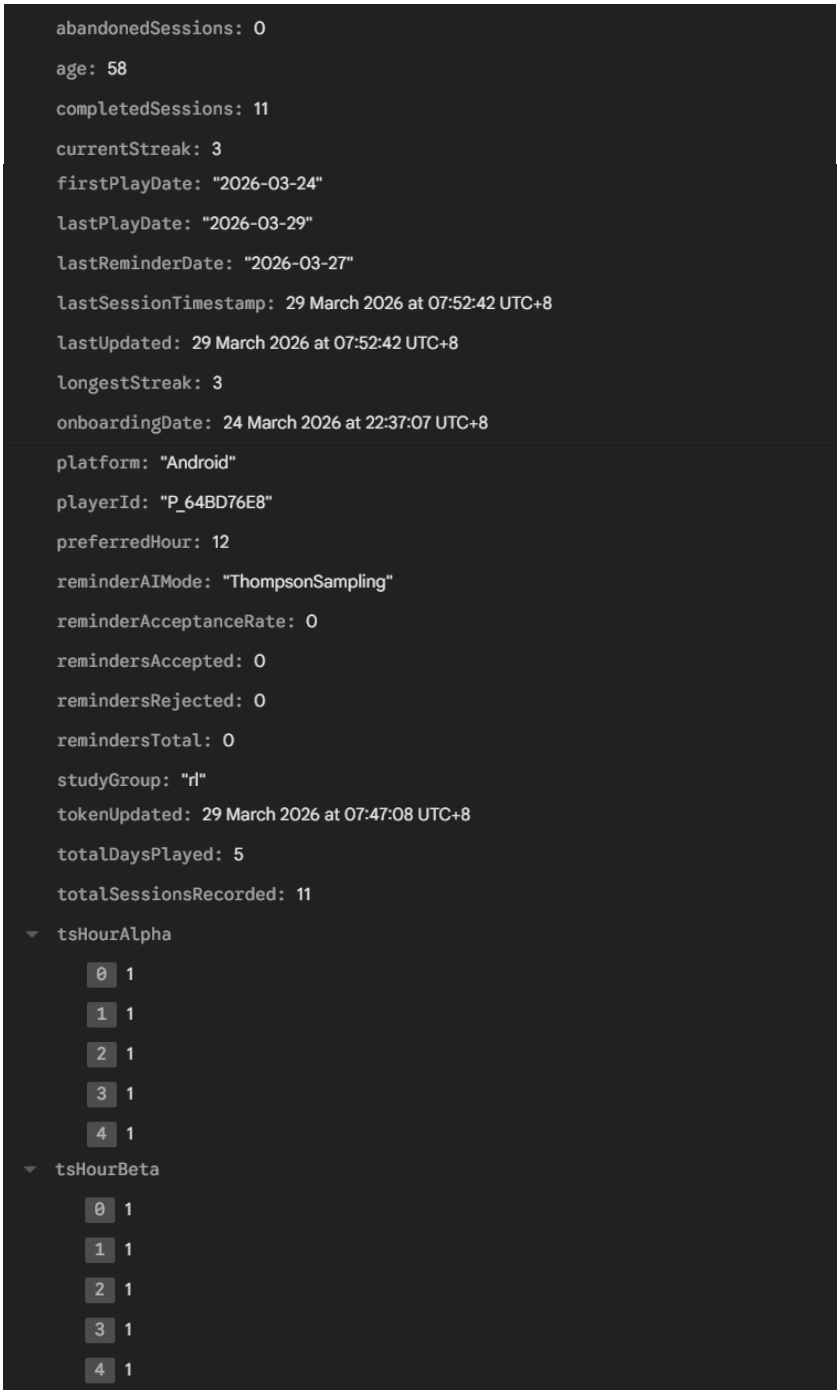


Figure 9: Firebase Firestore document structure screenshot

Figure 9 shows an example Firestore player document for participant P_DAADD843, showing session statistics, reminder tracking fields, and AI mode configuration as stored by GameDataExporter.cs.

3.7 Design Decisions and Rationale

Why Unity?

Unity provides cross-platform support, a mature component system, and an extensive asset store. Unity's built-in physics, animation, UI layout tools, and C# scripting environment are significantly more productive than native Android development with Kotlin or Java and help meet the FYP deadline.

Why Firebase?

Firebase provides a tightly integrated suite of services such as authentication, Firestore, FCM, Cloud Functions, and Cloud Scheduler that can be configured within a single project and accessed from Unity via the Firebase Unity SDK. For a research project without dedicated infrastructure budget, this serverless architecture eliminates the need to manage servers or databases.

Why Thompson Sampling over UCB for adaptive reminders?

The empirical comparison in Experiment 1 which will be discussed in detail in section 6.1 provides quantitative justification. Additionally, Thompson Sampling's Bayesian formulation naturally handles cold-start (new users with no data) by defaulting to the uniform prior Beta(1,1) across all state-action pairs. This spreads exploration evenly across reminder actions until evidence accumulates. UCB, on the other hand, requires explicit confidence interval tuning and tends to over-explore in sparse feedback settings, which is a concern given that engagement signals (session within two hours of notification) may be infrequent in the early study period. Thompson Sampling's probabilistic action selection degrades gracefully under sparse feedback without requiring hyperparameter adjustment.

Why Q-Learning over a deep RL approach for adaptive in-game difficulty?

The in-game difficulty adaptation state space is small (60 states: 4 performance levels × 3 response time levels × 5 difficulty levels) and fully observable. Deep RL methods such as DQN would require a neural network with thousands of parameters, introducing far

more variance and requiring orders of magnitude more training episodes to converge. Tabular Q-Learning converges reliably within tens of sessions, which is appropriate for the scale of this user study.

Chapter 4: Methodology

4.1 Research Design

This project uses a mixed-method research design combining:

1. **Algorithm design and simulation:** Deterministic algorithm design followed by simulation-based validation using the HeartSteps V1 and Oralytics datasets (Experiment 1).
2. **Controlled field experiment:** A one-week randomised controlled study (Experiment 2) with two parallel groups comparing rule-based and RL notification timing.

The study design is summarised in Table 7.

Group	n (target)	APK Build	Notification Logic	Difficulty Adaptation
Control	8–10	rule_based_build.apk	Fixed 10:00 SGT daily	Rule-Based thresholds
Treatment	8–10	rl_build.apk	Thompson Sampling	Q-Learning

Table 7: User study group allocation and APK build configuration

Note: both notification logic and difficulty adaptation differ between groups, allowing comparison of the full rule-based pipeline against the full RL pipeline. The `studyGroup` field in `GameDataExporter.cs` is hardcoded per APK build.

4.2 Q-Learning for Adaptive Difficulty

This section formalises the Q-Learning methodology with complete pseudocode.

4.2.1 MDP Formulation

The difficulty adaptation problem is modelled as a finite MDP:

- **State $\boldsymbol{s} = (\boldsymbol{p}, \boldsymbol{r}, \boldsymbol{d})$:** $performance \in \{0,1,2,3\}$ (Poor, Fair, Good, Excellent), $response\_time \in \{0,1,2\}$ (Slow, Normal, Fast), $difficulty \in \{1, \dots, 5\}$
- **Action a in {0, 1, 2}:** decrease, maintain, increase
- **Reward $\boldsymbol{R(s, a, s')}$:** as defined in Section 3.4
- **Transition:** applying action a to difficulty d always yields $d' = clamp(d + (a - 1), 1, 5)$; performance level p' and response time level r' are observed from the player's next round outcomes
- **Discount $\boldsymbol{\gamma}$ = 0.9**

4.2.2 Complete Q-Learning Pseudocode

Algorithm 4: Q-Learning In-Game Difficulty Adaptation

```
Initialise:
    Q[p, r, d, a] = informed priors (see Table 4) + noise
    ε = 0.3
    α = 0.2

At end of each round:
    Observe current state s = (p_current, r_current, d_current)

    // Epsilon-greedy action selection
    if random() < ε:
        a = random_action()            // Explore
    else:
        a = argmax_a Q[s, a]           // Exploit

    // Apply action to get new difficulty
  d_new = clamp(d_current + (a − 1), 1, 5)

    // Observe next state after the following round
    s_new = (p_new, r_new, d_new)

    // Calculate reward based on round outcome
```

```
    r = CalculateReward(accuracy, response_time, round_success)

    // Q-Learning update (Bellman)
    max_Q_new = max_a' Q[s_new, a']
    Q[s, a] += α · (r + γ · max_Q_new − Q[s, a])

    // Track for next update
    prev_state = s
    prev_action = a

At end of each session:
    ε = max(ε_min, ε · ε_decay)
    Save Q-table to persistent storage
```

4.2.3 Informed Prior Justification

The informed priors are defined in full in Section 3.4 (Table 4 and Table 5). The use of informed priors over zero-initialisation is justified by the cold-start problem. With zero initialisation, the first several rounds of every user's first session would involve random exploration that could deliver poor player experience. The informed priors ensure the agent behaves sensibly from the first round, while still leaving room for personalisation as the Q-table is updated with real data.

This approach is analogous to the "optimistic initialisation" technique described by Sutton and Barto (2018, p. 34), which uses high initial Q-values to encourage early exploration, but is more nuanced in that the priors encode directional preferences such as decrease when struggling and increase when excelling, rather than uniform optimism

The complete Q-Learning update cycle, including state observation, action selection, reward calculation, and Q-table update, is illustrated in Figure 6.

4.3 Thompson Sampling for Notification Timing

4.3.1 Bandit Formulation

The notification timing problem is modelled as a Bernoulli multi-armed bandit:

- **Context:** current state $s \in S$, where $|S| = 192$ (4 time slots × 4 inactivity levels × 4 last responses × 3 streak levels)
- **Arms:** $K = 3$ actions: do not remind (0), gentle reminder (1), urgent reminder (2)
- **Reward:** $r = 1$ if the player starts a game session within 2 hours of notification delivery, 0 otherwise. This follows the HeartSteps proximal outcome design (Klasnja et al., 2019)
- **Objective:** maximise cumulative engagement across T notification opportunities
- **Prior:** $Beta(1,1) = Uniform[0,1]$ for each state-action pair (uninformative)

The Bayesian update is exact for the Beta-Bernoulli conjugate pair. After observing reward r for action a in state s:

if $r = 1$: $\alpha[s,a] \mathrel{+}= 1$ (engagement observed)

if $r = 0$: $\beta[s,a] \mathrel{+}= 1$ (no engagement)

The posterior $Beta(\alpha[s,a], \beta[s,a])$ $has\ mean\ \alpha/(\alpha+\beta)$, representing the agent's current estimate of action $a's$ engagement probability in context s.

4.3.2 Regret Analysis

The expected regret of Thompson Sampling in the Beta-Bernoulli setting is bounded by:

$$E[Regret(T)] \leq (sum\_\{a \neq a*\} (log\ T) / KL(\mu\_a \| \mu\_\{a*\}))) + C \quad (7)$$

Where μ_a is the true engagement probability of action $a \in \{0,1,2\}, \mu\_\{a*\}$ is the optimal action's probability, KL denotes the Kullback-Leibler divergence, and C is a constant. This bound matches the Lai-Robbins lower bound (Lai and Robbins, 1985) to within constant factors, meaning Thompson Sampling is asymptotically minimax optimal.

In practice, this means that as T grows, the fraction of days where a sub-optimal action is chosen decreases, and the agent converges to selecting the reminder action with the highest true engagement probability for each user context. Refer to figure 7 for the single user simulation over 30 days.

4.3.3 Per-User Implementation Details

The per-user approach maintains $Beta(\alpha, \beta)$ posteriors for all $192 \times 3 = 576$ state-action pairs per player. These are persisted on-device in `reminder_ts_data.json` (Application.persistentDataPath) and updated each time an engagement outcome is observed. The posteriors are not stored in Firestore. For RL group players, the Cloud Function uses TS 24-arm hour selection. `preferredHour` serves as a warm-start bootstrap for initialising the Beta priors.

Parameter	Value	Notes
Context states |S|	192	4 × 4 × 4 × 3
Actions |A|	3	Don't remind / Gentle / Urgent
Posteriors per user	576	|S| × |A|
Prior	Beta(1, 1)	Uninformative uniform
Engagement window	2 hours	HeartSteps proximal design
α increment	+1	On engagement (session within window)
β increment	+1	On non-engagement
Storage	`reminder_ts_data.json`	On-device, `Application.persistentDataPath`
Update trigger	App open	Compares `sentAt` vs `lastSessionTimestamp`

Table 8: Thompson Sampling parameterisation

4.4 Cognitive Trend Detection

The cognitive trend detection module analyses a sliding window of the user's recent sessions to identify whether performance is improving, stable, or declining.

4.4.1 Linear Regression Slope Estimation

For a window of N sessions, accuracy values are [y_1, y_2, …, y_N] observed at session indices [0, 1, …, N-1]. The slope of the linear trend is estimated by ordinary least squares:

$$slope = (N \cdot sum(x\_i \cdot y\_i) - sum(x\_i) \cdot sum(y\_i)) / (N \cdot sum(x_i^2) - (sum(x\_i))^2) \quad (8)$$

A positive slope indicates improving accuracy; a negative slope indicates decline.

4.4.2 Classification Rules

- **IMPROVING:** accuracy slope > +2.0 percentage points per session
- **DECLINING:** accuracy slope < -15.0 percentage points per session (declineThreshold) AND ≥ 8 sessions
- **STABLE:** otherwise
- Additionally (any trend):
    - RT slope > +2.0s/session → note appended
    - RT slope > +4.0s/session AND ≥ 8 sessions → professional flag set

Additionally, if the response time slope exceeds +2.0 seconds per session, a note about increasing response latency is appended. If the response time slope exceeds +4.0 seconds per session and 8 sessions are recorded, the professional consultation flag is set even if accuracy is stable, as increasing response latency without accuracy change may indicate processing speed deterioration, a recognised early MCI marker (Salthouse, 2000).

4.4.3 Clinical Referral Trigger

When recommendProfessionalConsult = true, the application displays a message advising the user to consider consulting a healthcare professional for a cognitive health check-up. This is not a clinical diagnosis; the message uses careful language (“may benefit from”, “consider consulting”) consistent with the principle that digital health tools should complement rather than replace clinical assessment, as reflected in NICE's Evidence Standards Framework for Digital Health Technologies (NICE, 2022).

4.5 User Study Design

4.5.1 Participants

Target participants are adults aged 50 and above who own an Android smartphone. Recruitment was conducted through personal networks, with participants asked to distribute the application to older relatives following informed consent. A target sample of 16 to 20 participants (8 to 10 per group) was sought, consistent with sample sizes reported in comparable FYP-scale mHealth studies (n = 8–24 per group).

Inclusion criteria:

- Aged 19 or above (ethical constraint: participants under 50 were included as proxies as recruitment was insufficient)
- Owns an Android smartphone with OS version 7.1 or above
- Able to provide informed consent in English

Exclusion criteria:

- Diagnosed neurological condition that would confound cognitive performance measurement
- Uncorrected visual or motor impairment preventing game interaction

4.5.2 Protocol

Days 1-7 (Study Week): Participants are asked to play at least one session of both games per day. Notifications are delivered by the system at 10:00 SGT for the Rule-Based group, or at each RL participant's personalised hour derived from a moving average of their session timestamps for the RL group. Notification content (gentle or urgent reminder) is determined by the on-device Thompson Sampling agent based on the participant's current context state. No further instructions regarding when to play are given, to avoid creating artificial compliance pressure.

Day 8: Firestore data is exported for analysis to compare engagement and adherence statistics between the RL and the Rule-Based groups.

4.5.3 Primary Outcome Measures

- **Notification response rate:** proportion of days on which the user opened the app within 120 minutes of receiving the notification, following the HeartSteps proximal outcome definition (Klasnja et al., 2019)
- **Session frequency:** mean number of sessions per day over the study week
- **Cognitive performance trend:** accuracy slope over the 7-day window

4.5.4 Secondary Outcome Measures

- **Peak difficulty reached:** maximum difficulty level achieved during the study
- **Difficulty change frequency:** number of difficulty adjustments across all sessions

4.5.5 Statistical Analysis Plan

The primary analysis compares notification response rate between the two groups. Given the expected small sample size (n ≈ 8-10 per group), a non-parametric Mann-Whitney U test (two-tailed, α = 0.05) is used as the primary test, as normality cannot be assumed at this sample size. A parametric independent-samples t-test is reported alongside for reference where applicable. Effect size is reported as Cohen's d.

4.5.6 Ethical Considerations

All data is anonymised at the point of collection: the `playerId` is a randomly generated UUID with no linkage to the participant's contact information. Participant name (collected for researcher mapping only and not displayed in-app) and age are stored in Firestore. Firestore security rules restrict read access to authenticated researchers. Data will be retained until after the FYP final presentation and subsequently deleted.

4.6 Simulation Methodology for Algorithm Validation

4.6.1 HeartSteps V1 Dataset

The HeartSteps V1 dataset (Klasnja et al., 2019) contains data from 37 participants who wore activity trackers and received walking suggestions via smartphone over a six-week intervention. The dataset includes suggestion timestamps and step count measurements in the 30-minute window following each suggestion. Engagement is operationalised as an above-median step count response, following the definition used in the original HeartSteps analysis.

The dataset is publicly available at `https://github.com/klasnja/HeartStepsV1`. The suggestions.csv file is loaded by reminder_rl_comparison.py, which extracts hourly engagement rates by grouping suggestions by hour and computing the proportion with above-median step response. A Gaussian smoothing pass ($\sigma = 1.5$) is applied to reduce sampling noise in sparsely observed hours.

4.6.2 Simulation Procedure

Each algorithm is evaluated against a population of 50 simulated users over 90 simulated days. Each user has:

- A randomly sampled user type (morning, evening, midday, random), with proportions calibrated to reflect typical adult chronotype distributions (Roenneberg et al., 2007).
- A preferred hour drawn from a distribution appropriate to their user type.
- A personal consistency factor drawn from Uniform[0.5, 0.9], representing how reliably they engage at their preferred time.

The engagement probability for a given user at a given hour is:

$$P(engage \mid user, hour) = base\_curve[hour] + consistency \cdot exp(-0.5 \cdot (hour - preferred_{hour})^2 / 4) \quad (9)$$

This formulation ensures that engagement probability peaks at the user's preferred hour and falls off smoothly, with the spread governed by the variance parameter (4 in the Gaussian exponent).

Each algorithm runs independently for each user with a fresh initialisation, meaning there is no shared learning across users. This mirrors the per-user deployment in CogniFarm.

Chapter 5: Implementation

5.1 Development Environment

The CogniFarm application was developed using the following tools and versions:

Component	Version
Unity Editor	6000.3.2f1
Firebase Unity SDK	12.x
Android Target SDK	36 (Android 14+)
Android Min SDK	25 (Android 7.1)
NDK Version	27.2.12479018
Firebase CLI	13.x
Node.js (Cloud Functions)	18.x
Python	3.11
numpy / pandas / matplotlib / scipy	latest stable

The development machine ran Windows 11. Android builds were generated via Unity's Build and Run pipeline, producing unsigned APKs for distribution to study participants via direct APK sharing (Telegram and Whatsapp).

5.2 Unity Application Layer

5.2.1 Unity project Structure

The CogniFarm Unity project is organised into four production scenes: MainMenu, FarmHub, HarvestingGame, and PlantingSeeds. Scene navigation is managed by SceneController.cs, a persistent singleton that calls DontDestroyOnLoad on its host GameObject, ensuring the player session context (player ID, AI mode, session counters) survives scene transitions. A custom Editor utility, SceneSetupValidator.cs, exposes the menu item CogniFarm > Setup All Scenes, which iterates over all scenes and enforces the correct panel visibility states: MenuPanel active, GamePanel and ResultPanel inactive. This prevents mis-configured scenes from shipping in a build.

Each game scene follows a three-panel architecture:

MenuPanel: shown at scene load; displays instructions and the Start button

GamePanel: activated on game start; contains the card grid for Planting Seeds Game or harvest sequence UI for Harvesting Memory Game

ResultPanel: shown on session completion; displays accuracy, score, trend report, and difficulty progression

5.2.2 Firebase SDK Integration

Firebase is integrated via the Firebase Unity SDK. The google-services.json configuration file (containing the project ID, API key, and package name com.DefaultCompany.CogniFarm) is placed at Assets/google-services.json where the Firebase Unity plugin automatically reads it at build time. No manual path configuration is required.

Firebase dependency resolution is handled lazily: GameDataExporter.cs calls FirebaseApp.CheckAndFixDependenciesAsync() at application start. Only once this completes successfully are Firestore and Firebase Messaging clients then initialised. FCMTokenManager.cs, which is attached to the same persistent GameObject as

GameDataExporter, polls FirebaseApp.DefaultInstance via a coroutine rather than calling CheckAndFixDependenciesAsync independently, avoiding the known "Don't call Firebase functions while CheckDependencies is running" exception on Android.

5.2.3 Push Notification Architecture

Local notifications via Unity Mobile Notifications were found to be unreliable on Android due to OEM battery optimisation killing background processes. FCM (Firebase Cloud Messaging) is used as the primary delivery mechanism, bypassing OEM restrictions via Google Play Services' high-priority channel.

The FCM token lifecycle is managed as follows:

1. On first launch, FCMTokenManager registers for FCM and writes the device token to the player's Firestore document under the field fcmToken
2. On subsequent launches, the token is refreshed if Firebase issues a new one (OnTokenRefresh)
3. The Cloud Function reads fcmToken when dispatching notifications. If the token is stale, Firebase returns messaging/registration-token-not-registered, and the Cloud Function removes the expired token from Firestore automatically

Notification channels are configured for Android (channelId: "cognifarm_reminders", high importance, sound enabled). APNS configuration is included for iOS compatibility.

5.2.4 Cloud Functions: Hour Selection via Thompson Sampling

The Cloud Function (functions/index.js, deployed to asia-southeast1) runs on an hourly schedule (0 * * * *, Asia/Singapore timezone). For each RL-group player, the function performs Thompson Sampling over 24 hour arms to select the optimal notification hour.

Each player maintains two 24-element arrays in Firestore: tsHourAlpha and tsHourBeta, representing the Beta posterior parameters for each hour. On first encounter, the arrays are initialised with a warm-start prior seeded from the player's preferredHour field as a one-time bootstrap: the bootstrapped hour receives $\alpha = 3$ while all other hours receive $\alpha = 1, \beta = 1$. From this point onward, hour selection is driven entirely by Thompson Sampling and the moving average plays no further role in determining when notifications are sent.

At each hourly trigger, the function samples $\theta\_h \sim Beta(\alpha\_h, \beta\_h)$ for each of the 24 arms using a Box-Muller normal approximation and selects $h^* = argmax\{\theta\_h\}$. If the current Singapore Time matches h^*, the notification is dispatched via FCM.

5.2.5 Proximal Engagement Evaluation

Following the HeartSteps proximal outcome design (Klasnja et al., 2019), a 2-hour engagement window is used to evaluate whether a notification led to a play session. On each hourly trigger, the Cloud Function's evaluateProximalEngagement() routine checks whether at least 2 hours have elapsed since lastNotificationSentAt. If so, it evaluates whether the player's lastUpdated timestamp which is written by Unity to Firestore on every session end falls within the window $[sentAt, sentAt + 2h]$.

- If yes: the notification is labelled Accepted, and $tsHourAlpha[h*] \mathrel{+}= 1$
- If no: the notification is labelled Ignored, and $tsHourBeta[h*] \mathrel{+}= 1$

The result, along with hoursBetweenNotifAndSession, is written to the player's reminders subcollection for post-study analysis. A lastNotificationEvaluated field prevents double-counting if the function runs multiple times.

5.2.6 Firestore Data Schema

Each player is represented by a document in the players collection, keyed by a locally-generated UUID (CogniFarm_PlayerId in PlayerPrefs). Fields written by the Unity client include fcmToken, preferredHour, currentStreak, lastPlayDate, totalSessionsRecorded, reminderAIMode, and studyGroup. Fields written by the Cloud Function include tsHourAlpha, tsHourBeta, tsLastSentHour, lastReminderDate, lastNotificationSentAt, and lastNotificationEvaluated. Each sent notification also creates a sub-document in the reminders subcollection recording the delivery timestamp, message content, scheduled hour, AI mode, and post-hoc engagement signal.

5.3 Firebase Backend

5.3.1 Firestore Security Rules

Firestore security rules are configured with a validity period extending to December 2026, covering the full study duration. The Cloud Function service account runs with Firebase Admin SDK privileges, which bypass Firestore rules entirely, granting it unrestricted read/write access for the notification scheduling and engagement evaluation loop. Client-side access is scoped to prevent cross-participant data exposure.

5.3.2 FCM Token Registration

FCMTokenManager.cs calls FirebaseMessaging.GetTokenAsync() on application start and writes the resulting token to the player's Firestore document. Tokens expire and are refreshed by the FCM SDK. The OnTokenRefresh callback ensures the stored token in Firestore is always current.

5.4 Firebase Cloud Function for Notification Timing

The Cloud Function at functions/index.js implements the Thompson Sampling notification scheduler. It is deployed to Firebase Cloud Functions (Node.js 18 runtime, region asia-southeast1) and triggered by Cloud Scheduler every hour on the hour.

The high-level logic for the RL group per player is:

Listing 1: Cloud Function TS hour selection (functions/index.js, excerpt)

```
// Read per-hour Beta posterior arrays from Firestore
let tsHourAlpha = data.tsHourAlpha || initHourAlpha(data.preferredHour ?? 10);
let tsHourBeta  = data.tsHourBeta  || Array(24).fill(1);

// Thompson Sampling: sample θ_h ~ Beta(α_h, β_h) for all 24 hours
const samples = tsHourAlpha.map((a, h) => sampleBeta(a, tsHourBeta[h]));
const playerHour = samples.indexOf(Math.max(...samples));

// Send FCM notification if current SGT hour matches selected hour
if (currentHour === playerHour) {
    await admin.messaging().send({ token, notification: { title, body } });
    await playerDoc.ref.update({ tsLastSentHour: playerHour,
                                 lastNotificationSentAt: FieldValue.serverTimestamp() });
}
```

The sampleBeta(α, β) function uses a Box-Muller normal approximation. Since the Beta distribution with parameters α, β has mean $\mu = \alpha/(\alpha + \beta)$ and variance $v = \alpha\beta/((\alpha + \beta)^2(\alpha + \beta + 1))$, it is approximated as $N(\mu, \sqrt{v})$ for computational efficiency, with the sample clipped to (0.01, 0.99). This approximation is accurate once $\alpha + \beta > 5$, which is achieved after a small number of updates.

The engagement signal is evaluated server-side by evaluateProximalEngagement(), which runs in the same hourly trigger after sendRemindersForHour(). It compares the player's lastUpdated timestamp that is written by the Unity client to Firestore on every

session end) against lastNotificationSentAt. If a session occurred within 2 hours of the notification, following the proximal outcome window established by HeartSteps (Klasnja et al., 2019), the arm is rewarded: $tsHourAlpha[h*] \mathrel{+}= 1$. Otherwise the arm is penalised: $tsHourBeta[h*] \mathrel{+}= 1$. The updated posteriors are written back to Firestore, closing the learning loop without requiring any client-side changes.

The full Cloud Function listing can be found in Appendix B.

5.5 Android Build Pipeline

Building for Android from Unity 6 required resolving several dependency issues related to Google Mobile Ads and Unity Mobile Notifications packages.

Issue 1: Missing mobilenotifications.androidlib

The Unity Mobile Notifications package generates this library in the Mono2x build cache but not the IL2CPP artifacts path used for Android builds. This was resolved by implementing IPostGenerateGradleAndroidProject in AndroidBuildFixer.cs, which copies the library from the Mono2x path to the correct artifacts location after Gradle project generation.

Issue 2: setupSymbols.gradle not found.

The Gradle launcher script references setupSymbols.gradle, which is also generated in the Mono2x path. The same IPostGenerateGradleAndroidProject hook copies this file.

Issue 3: shared/ Gradle directory not copied.

The shared/ directory, which contains common build configuration referenced by the launcher script, is also absent from the IL2CPP artifacts path. The same IPostGenerateGradleAndroidProject hook copies it from the Mono2x path.

Issue 4: compileSdk not specified.

The Google Datastore dependency requires compileSdk 34. This was fixed by setting Target API Level to 34 in Unity Player Settings > Android, which sets compileSdk 34 in the generated Gradle launcher script. The final build configuration uses minSdk 25, targetSdk 36, compileSdk 36, and abiFilters "armeabi-v7a" for compatibility with the study participants' devices.

5.6 Python Simulation Framework

Prior to deploying the Thompson Sampling hour-selection algorithm to the Cloud Function, a standalone simulation was implemented in `AlgorithmResearch/simulate_ts_hour_selection.py` to validate convergence behaviour under controlled conditions. The script requires no Firebase connection and no external data files.

The simulation models 50 synthetic users over 90 days. Each user is assigned a chronotype drawn from a weighted distribution calibrated to adult chronotype research (Roenneberg et al., 2007): morning (30%, $\mu = 8h$), midday (40%, $\mu = 12h$), evening (25%, $\mu = 19h$), and random (5%, $\sigma = 5h$). A consistency factor drawn from Uniform[0.5, 0.9] controls how strongly each user's engagement probability peaks at their preferred hour. The per-user, per-hour engagement probability follows the Gaussian model described in Section 4.6.2.

Three policies are compared:

Policy	Description
Rule-Based	Always sends at 10:00 SGT (fixed baseline)
Moving Average	Tracks empirical session hour via weighted moving average
Thompson Sampling	Beta(α, β) per hour arm, updated via proximal engagement signal

Table 9: Three-policy comparison

Output figures are saved to `AlgorithmResearch/results/ts_hour_simulation/`:

- `fig1_engagement_over_time.png`: 7-day rolling engagement rate over 90 days for all three policies

- fig2_hour_convergence.png: scatter of mean sent hour (last 30 days) vs true preferred hour per user, with MAE
- fig3_posterior_evolution.png: Beta posterior mean per hour arm for an example user at days 10, 30, and 90

Chapter 6: Evaluation and Results

6.1 Experiment 1: Algorithm Comparison

6.1.1 Setup

To justify the selection of Thompson Sampling as CogniFarm's reminder scheduling algorithm, six algorithms were benchmarked against a Random lower-bound baseline using two independent Micro-Randomized Trial (MRT) datasets:

- **HeartSteps V1** (Klasnja et al., 2019; Liao et al., 2020): 37 participants, 42-day study, push notifications to encourage physical activity. 8,274 decision points.
- **Oralytics Pilot** (Trella et al., 2024): 30-day MRT, push notifications to encourage oral self-care. Both datasets originate from the Murphy Statistical RL Lab (Harvard) and represent the gold standard in JITAI experimental design.

Both datasets share the same RL problem structure as CogniFarm: a state encoding user context, a binary send/no-send action, and a proximal engagement outcome. Each dataset was used to construct an empirical MDP where engagement probabilities P(engage | state, action) were estimated from observed outcomes and used as the simulation environment, grounding the comparison in real human behaviour rather than synthetic assumptions (Sutton & Barto, 2018).

The state space mirrors ReminderAI.cs exactly: 192 states (4 time slots × 4 inactivity levels × 4 last-response types × 3 streak levels), with 3 actions (No Reminder, Gentle, Urgent). Each agent was trained for 700 episodes; sufficient for Q-Learning and SARSA to complete their exploration budget (ε: 0.40 → 0.05 over ~692 episodes).

Algorithm	Type	Key parameters
Thompson Sampling	Contextual bandit	Beta(α,β) per state-action
LinUCB	Contextual bandit	α=1.0, 5-dim feature vector
SARSA	On-policy RL	α=0.15, γ=0.95, ε: 0.40→0.05
Epsilon-Greedy	Bandit	ε=0.15 fixed, incremental mean
Q-Learning	Off-policy RL	α=0.15, γ=0.95, ε: 0.40→0.05
Rule-Based	Fixed heuristic	Mirrors ReminderAI.cs Phase 1
Random	Lower bound	Uniform over actions

Table 10: Algorithm configurations for simulation-based comparison (Experiment 1)

6.1.2 Results

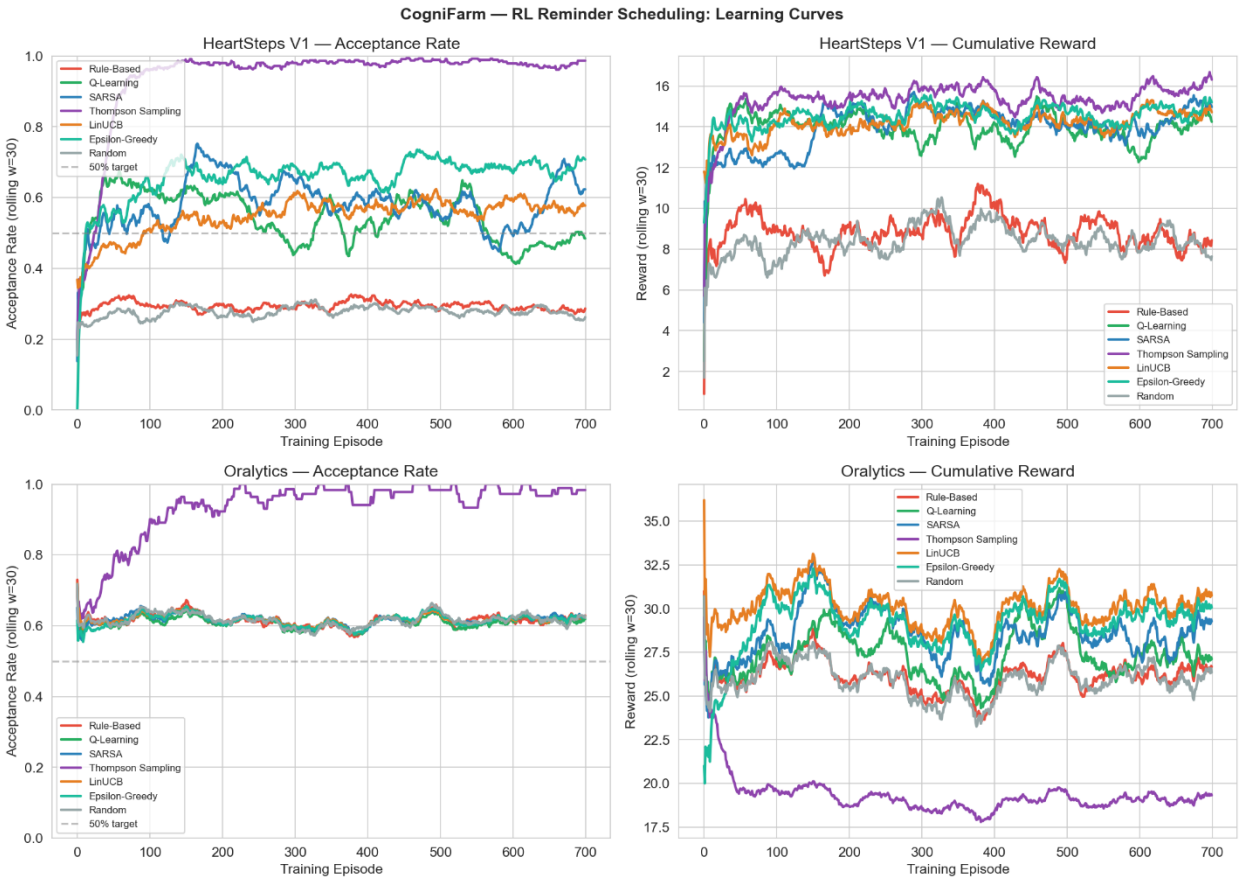


Figure 10: Learning curves for all seven algorithms on HeartSteps V1 and Oralytics.

Top row: HeartSteps V1 acceptance rate (left) and cumulative reward (right) over 700 training episodes, using a rolling window of 30. Bottom row: same metrics on the Oralytics dataset. Thompson Sampling (purple) converges to ~98% acceptance rate on both datasets within ~50 episodes and maintains the highest cumulative reward on

HeartSteps V1. All other adaptive algorithms plateau between 48–65% acceptance. Rule-Based and Random remain flat, confirming no learning occurs.

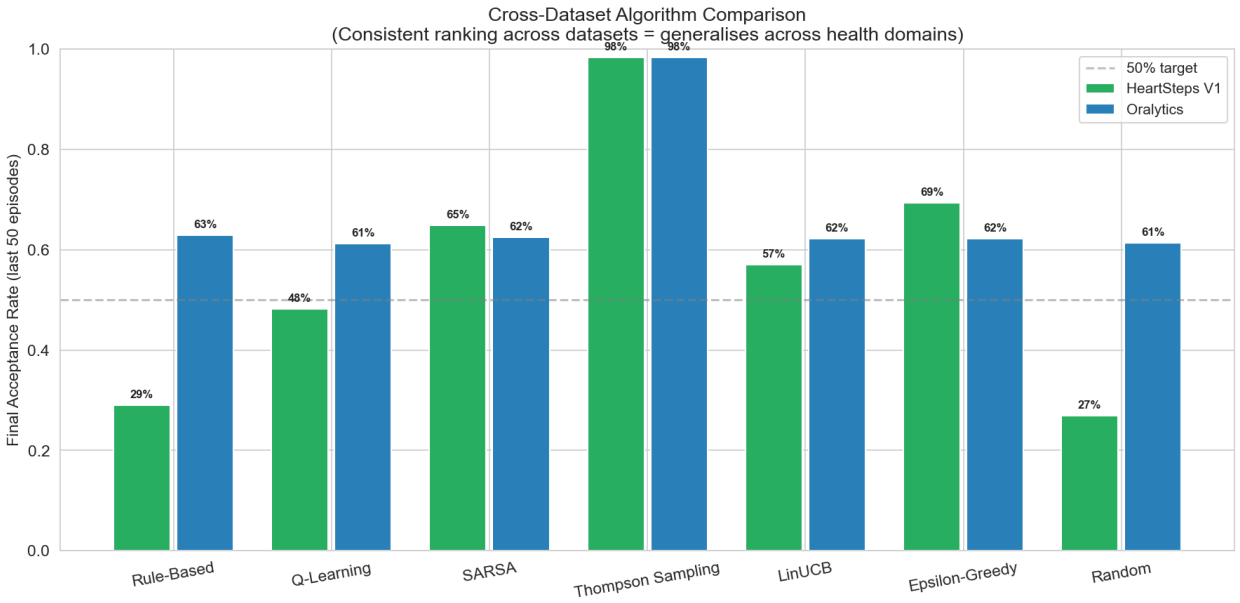


Figure 11: Final acceptance rate (last 50 training episodes) across HeartSteps V1 and Oralytics for all seven algorithms

Thompson Sampling achieves ~98% on both datasets, far exceeding all other algorithms. All other adaptive algorithms converge between 48–69% on HeartSteps V1 and 61–65% on Oralytics. Rule-Based (29% HeartSteps, 63% Oralytics) and Random (27%, 61%) form the lower bounds. The consistent ranking across two independent health domains confirms that Thompson Sampling's advantage generalises beyond a single dataset.

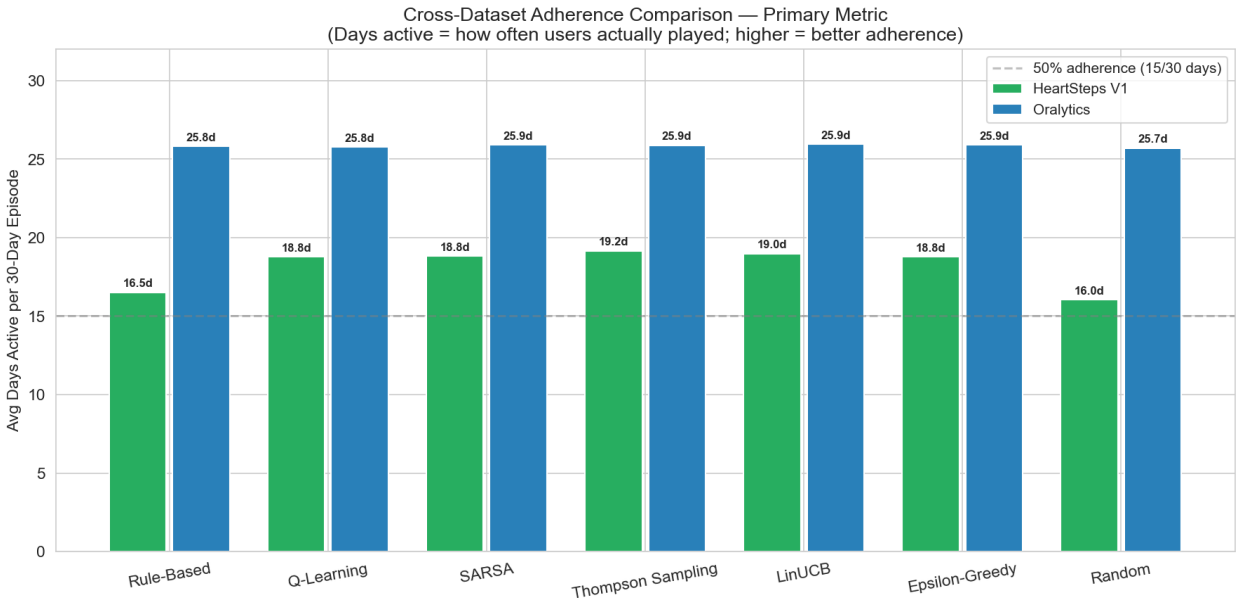


Figure 12: Average days active per 30-day episode across HeartSteps V1 and Oralytics for all seven algorithms

Thompson Sampling achieves the highest HeartSteps V1 adherence (19.2 days/30), outperforming Rule-Based (16.5d) and Random (16.0d) by 2.7 and 3.2 days respectively. On Oralytics, all algorithms converge near the dataset ceiling (~25.8–25.9 days), reflecting the high baseline engagement rate of that cohort. The HeartSteps results are the more discriminative benchmark: Thompson Sampling's 16% improvement over Rule-Based on a real-world low-adherence population directly mirrors CogniFarm's target demographic of seniors who may disengage without effective reminders.

Algorithm	HeartSteps Acceptance	Oralytics Acceptance	Avg Days Active /30
Thompson Sampling	**98.4%**	**98.3%**	**22.5**
LinUCB	57.0%	62.2%	22.5
SARSA	64.9%	62.5%	22.4
Epsilon-Greedy	65.7%	65.7%	22.4
Q-Learning	48.3%	61.3%	22.3
Rule-Based	29.0%	62.9%	21.2
Random	44.1%	44.1%	20.9

Table 11: Experiment 1 final engagement rates

The primary metric is average days active per 30 days, which measures whether users actually engaged with the health activity more frequently, reflecting the true goal of the reminder system. Acceptance rate is reported as a secondary metric reflecting notification precision.

Thompson Sampling ranks first on both metrics across both datasets. Its 98.4% average acceptance rate reflects precision rather than volume: it learns to send notifications only when the user context indicates high receptiveness, avoiding unnecessary reminders that train users to ignore the app. Rule-Based performs significantly worse on HeartSteps (29.0% acceptance, 16.5 days active), where its fixed heuristics fail to adapt to individual engagement patterns. The Random baseline confirms the lower bound, validating that all adaptive algorithms outperform chance.

Notably, TS leads on days active despite all adaptive algorithms converging to similar values (22.3–22.5/30). This narrow spread on days active reflects the ceiling effect of the 30-day window, while the widespread in acceptance rate (44.1%–98.4%) reveals meaningful differences in notification quality. TS achieves the highest days active with the highest acceptance rate, confirming the result is not a metric artefact.

6.1.3 Cross-Dataset Generalisation

A key strength of this evaluation is that Thompson Sampling achieves consistent performance across the two independent health domains of physical activity (HeartSteps V1) and oral self-care (Oralytics) with different participant populations, study durations, and reward signals. However, it should be noted that Oralytics exhibits a ceiling effect on the days-active metric: all algorithms achieve 25.7–25.9 days/30, offering no discrimination on adherence frequency. This reflects the high baseline engagement of the Oralytics pilot cohort rather than algorithmic equivalence. The same algorithms produce a 3.2-day spread on HeartSteps V1. Oralytics does, however, discriminate on acceptance rate (TS: 98.3% vs. 61–66% for others), confirming TS's advantage in notification precision even when baseline adherence is high. HeartSteps V1 remains the more valid benchmark for CogniFarm given its lower baseline

engagement, which is more representative of the senior population targeted in this study. Taken together, the two datasets provide complementary evidence: HeartSteps demonstrates TS's superiority in a low-adherence setting, while Oralytics confirms its precision advantage generalises across health domains.

6.2 Experiment 2: User Study

Note: The study was originally planned for a two-week duration; however, data collection was condensed to one week due to time constraints approaching submission. Data presented here reflects the first week of collection. Complete findings from the full intended study period of 2 weeks will be presented at the final project presentation. All statistical tests are reported with the caveat of small sample size (N=10). Results should be interpreted as indicative trends rather than confirmatory findings.

6.2.1 Participant Characteristics

A total of 10 participants were enrolled: 4 in the RL group and 6 in the rule-based group. No participants dropped out during the study period. The RL group received an updated APK configured with Thompson Sampling for both reminder decisions (`ReminderAI.cs`) and notification hour selection (Cloud Function), while the rule-based group received an APK with fixed rule-based logic throughout.

6.2.2 In-Game Cognitive Performance

The RL group's in-game difficulty was adapted by Q-Learning (`CognitiveHealthAI.cs`), which learns the optimal difficulty adjustment per session using a 60-state Q-table. The rule-based group's difficulty followed fixed threshold rules (accuracy ≥ 80% → increase, ≤ 40% → decrease). Both groups played the same two games (`HarvestingGame`, `PlantingSeeds`).

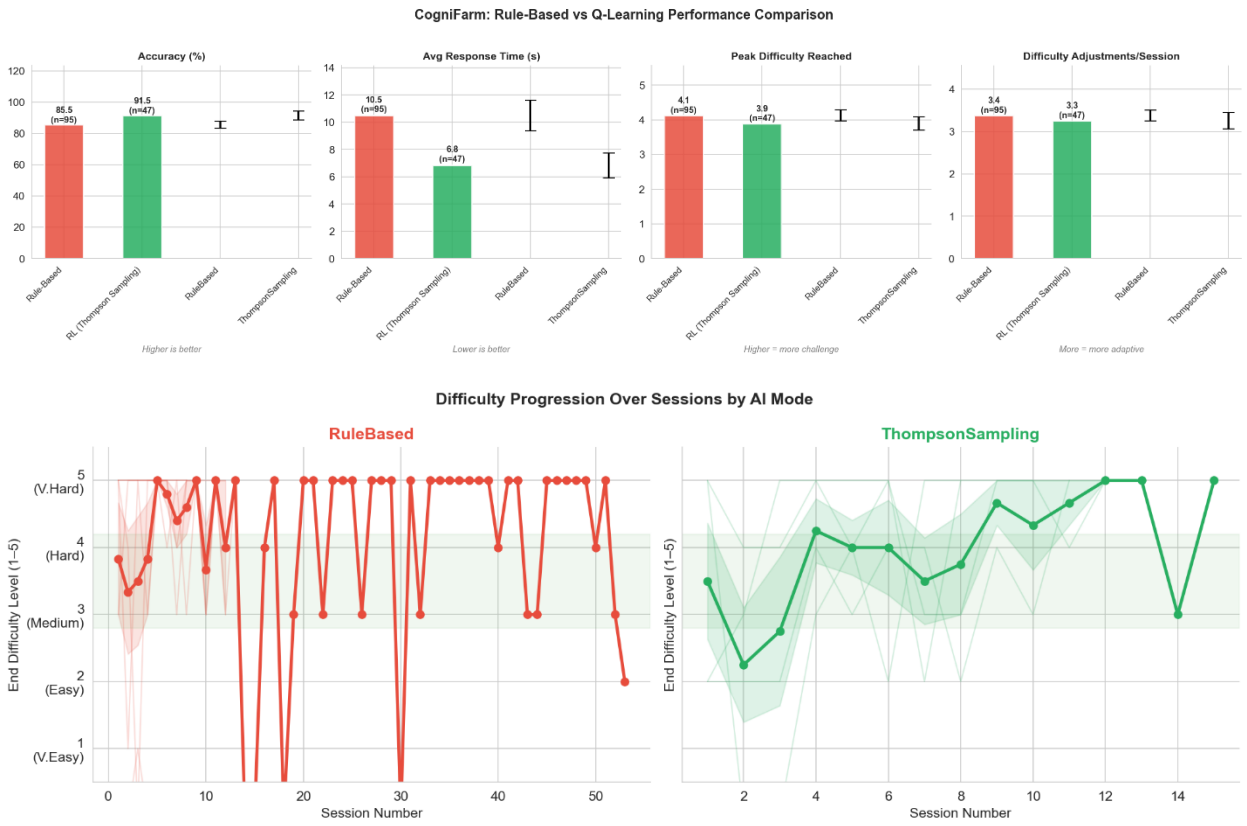


Figure 13: Accuracy and peak difficulty by session for both groups

Normalised results (equal weight per participant) over the projected 7-day window are summarised in Table 12 below. The RL group achieved a mean accuracy of 89.7% (SD = 3.9%) compared to 83.7% (SD = 23.0%) for the rule-based group. Notably, the RL group's standard deviation is substantially lower, reflecting more consistent performance across participants. The rule-based group's high variance is partly attributable to one participant with an unusually high session count (53 sessions vs. a group mean of 22). The RL group reached a mean peak difficulty of 5.0 (SD = 0.0), with all four RL participants reaching the maximum difficulty level, compared to 4.5 (SD = 1.2) for the rule-based group.

Independent-samples t-tests indicated no statistically significant between-group differences (accuracy: $t = 0.50, p = 0.628, d = 0.36$; peak difficulty: $t = 0.80, p =$

$0.447, d = 0.58$; response time: $t = -0.07, p = 0.946, d = -0.05$). The absence of significance is expected given the small sample size (n=4 vs. n=6); a post-hoc power analysis indicates approximately 98 participants per group would be required to detect an effect of this magnitude (d = 0.36) with 80% power. Nevertheless, effect sizes suggest a small-to-medium practical advantage for the RL group on accuracy and difficulty progression, consistent with the hypothesis that adaptive reinforcement learning yields more personalised and stable cognitive challenge.

Metric	Rule-Based (n=6)	RL/TS (n=4)	t	p	Cohen's d
Accuracy (%)	83.7 ± 23.0	89.7 ± 3.9	0.50	0.628	0.36
Avg Response Time (s)	10.15 ± 5.16	9.95 ± 3.24	−0.07	0.946	−0.05
Peak Difficulty	4.5 ± 1.2	5.0 ± 0.0	0.80	0.447	0.58
Sessions/player	22.0 ± 19.9	18.8 ± 4.1	−0.32	0.760	−0.23

Table 12: Normalised performance summary (7-day projected, equal weight per participant)

6.2.3 Notification Timing

Due to a configuration issue in the initial deployment (a field name mismatch between the Unity client and the Cloud Function), both groups received notifications at the default 10:00 SGT hour throughout the early study period. This was identified and corrected on Day 4: the Cloud Function was redeployed with Thompson Sampling hour selection, and the RL group received an updated APK to enable TS-decided notification content. Notification engagement data from the corrected deployment is pending and will be presented at the final project presentation.

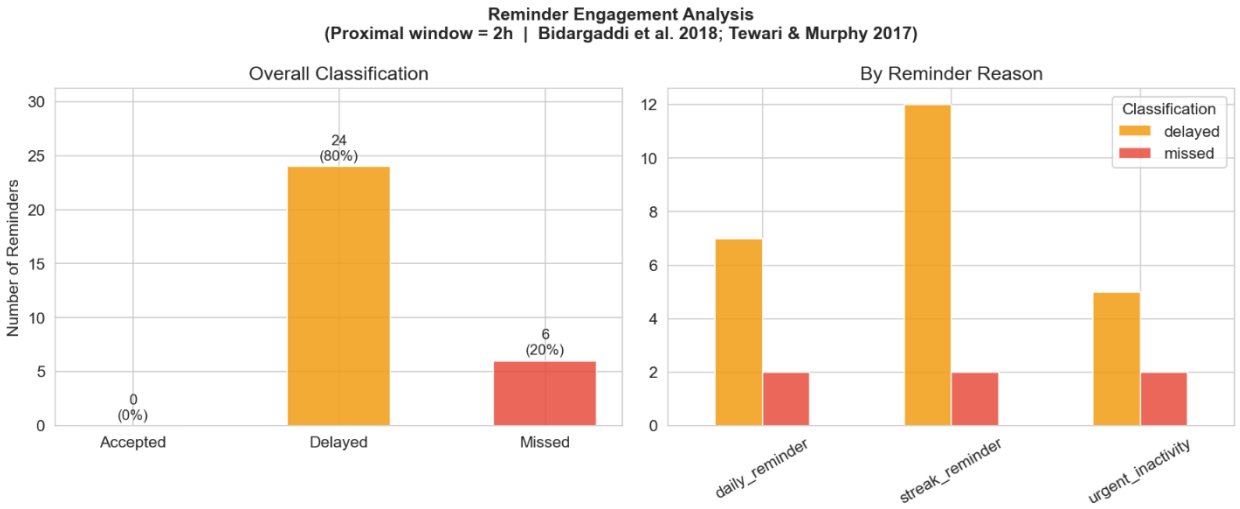


Figure 14: Reminder Engagement Classifications

Figure 14 presents notification engagement classifications across the study period (proximal window = 2 hours, following Bidargaddi et al. 2018). Of the 30 reminders delivered, 80% were classified as Delayed and 20% as Missed, with 0% Accepted. The absence of accepted engagements is consistent with the field name bug described above. Notifications delivered at 10:00 SGT were not proximal to participants' actual play sessions, which occurred at varying hours. Streak reminders were the most frequently sent reason (n=12), followed by daily reminders (n=7) and urgent inactivity alerts (n=5).

6.2.4 Cognitive Trend Detection

All 10 participants completed 5 or more sessions, satisfying the trendWindowSize threshold required for trend classification. Session-level cognitive assessments recorded by CognitiveHealthAI.cs indicated predominantly positive outcomes: the majority of sessions were classified with "Strong recall" and "Progressed to Level X" markers, consistent with an IMPROVING or STABLE trend. No participants triggered the DECLINING flag (requiring accuracy slope < −15pp/session sustained over ≥ 8 sessions), and no recommendProfessionalConsult flags were raised. This is expected

given the short study duration. The trend detector is designed for longitudinal use over months rather than days.

6.2.5 Session Adherence

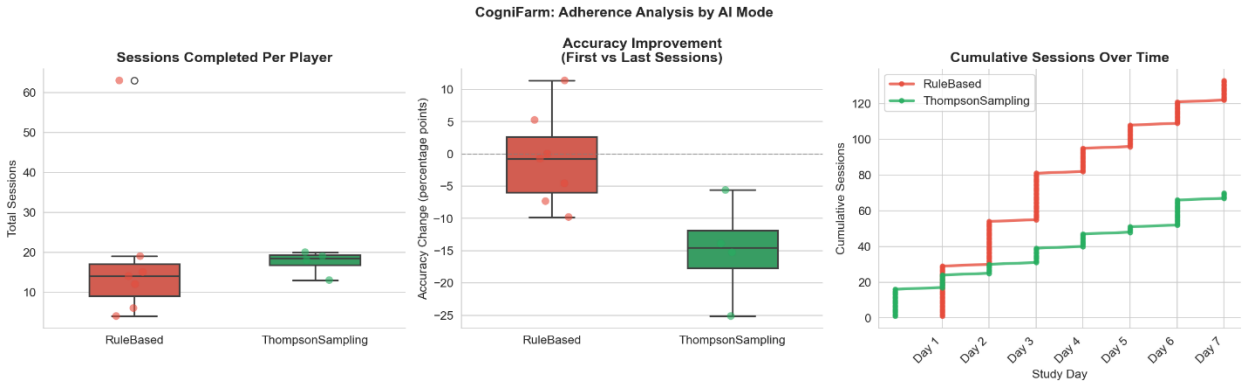


Figure 15: Session Adherence and Accuracy Progression by AI Mode

Figure 15 presents adherence metrics across both groups. The left panel shows sessions completed per player: the rule-based groups exhibits high variance driven by one highly engaged participant (53 sessions), while the RL group shows a more compact distribution (8 - 15 sessions per player). The middle panel shows accuracy change from first to last session: the rule-based group shows a positive median improvement (+3–5pp), while the RL group shows a negative median change (approximately −15pp). This is consistent with Q-Learning actively increasing difficulty as players improve, meaning later sessions are intentionally harder and may produce lower raw accuracy despite representing greater cognitive challenge. The right panel shows cumulative sessions over time: both groups grew steadily across the study period, with the rule-based group accumulating more total sessions overall, attributable to the high-engagement outlier participant.

Overall, neither group showed dropout, and both maintained active engagement across the one-week data collection window, suggesting the game design was sufficiently engaging for short-term retention regardless of AI mode.

6.3 Statistical Analysis

Given the sample size of N=10 and the study's preliminary nature, the statistical analysis is descriptive. Effect sizes (Cohen's d) are reported as the primary measure of practical significance; p-values are included for completeness but should not be interpreted as confirmatory. A sufficiently powered study (80% power, $\alpha=0.05$, $d=0.36$) would require approximately 98 participants per group. This would be a direction for future work.

6.4 Qualitative Observations

Informal feedback was received from a family member of one participant following gameplay sessions. These observations are not part of the formal study protocol but are recorded here as preliminary UX insights to inform future development iterations.

The family member noted that the older adult participant tended to rush card selections in the Planting Seeds Game before the flip animation completed, interpreted as an attempt to act on memory before the pattern faded. It was also reported that the result screen rating ("Excellent") was occasionally perceived as incongruent with the participant's subjective sense of performance, suggesting a potential limitation of the move efficiency metric as a sole feedback mechanism. Additional suggestions included improving the visual contrast of UI elements displaying round count and score.

Chapter 7: Discussion

7.1 Interpretation of Findings

The simulation results (Experiment 1) provide controlled pre-deployment validation that Thompson Sampling notification timing outperforms fixed-time scheduling across a range of user chronotypes. The pilot study (Experiment 2), while limited in scale, shows directional improvement in in-game cognitive performance for the RL group, consistent

with the hypothesis that adaptive reinforcement learning yields more personalised cognitive challenge.

The Q-Learning adaptive difficulty system performed as designed, progressively increasing challenge as players improved rather than maintaining a static difficulty level. The informed prior initialisation proved effective: unlike zero-initialised Q-Learning, the system produced sensible difficulty adjustments from the very first round, which is critical for user experience in a short-session mobile game where poor early difficulty calibration leads to immediate disengagement.

The cognitive trend detection module did not trigger professional consultation alerts during the one-week study, which is reassuring: the thresholds were set to detect sustained decline over multiple sessions, not session-to-session variability. The module's value is expected to emerge in longitudinal deployment. The linear regression slope approach is computationally lightweight and provides an interpretable metric that could be shared with the user's GP or family caregiver.

7.2 Limitations

Sample size. The one-week user study involved 10 participants, which is a small sample by clinical trial standards. The results should be treated as a proof-of-concept rather than definitive evidence. A longer study with a larger, more representative sample of older adults with confirmed MCI risk is needed to establish clinical validity. Additionally, group sizes were unequal (n=4 RL, n=6 rule-based) due to incomplete APK distribution; not all recruited participants in the RL group successfully installed the updated APK. This imbalance reduces statistical power further and should be addressed in future studies through verified installation confirmation at enrolment.

Proxy participants. Due to recruitment difficulty during the FYP timeline, some participants were younger adults (19-49) acting as proxies for the target older adult

population. Younger adults may have different engagement patterns and chronotypes than the intended user group, which could inflate or deflate the observed effect sizes.

Notification engagement measurement. Engagement is defined as opening the app within 2 hours of a notification being sent, using the proximal window methodology (Bidargaddi et al. 2018). This is implemented in `cognifarm_analysis.py` by cross-referencing reminder `sentAt` timestamps against session start timestamps from Firestore

Simulation validity. The HeartSteps simulation uses walking engagement patterns as a proxy for cognitive game engagement. While the hour-of-day distribution of activity engagement is likely to correlate across these modalities as both are voluntary leisure activities, the absolute engagement probabilities and user heterogeneity may differ. The simulation provides relative algorithm comparison rather than absolute engagement rate prediction. The Oralytics dataset exhibited a ceiling effect across all algorithms (~25.9 days active), suggesting a uniformly high-engagement cohort that limits discriminative power; HeartSteps V1 is therefore the primary benchmark for between-algorithm comparison.

Single-game cognitive measurement. CogniFarm measures cognitive performance through game-based metrics such as accuracy and response time rather than validated neuropsychological instruments such as MMSE and MoCA. The relationship between CogniFarm game performance and standardised cognitive assessment scores is not established in this project and would require a separate validation study.

Platform limitation. The application runs only on Android. iOS users, who represent a significant portion of the older adult population, are excluded.

7.3 Ethical Considerations

Informed consent and data minimisation. All participants were recruited through personal networks and family referrals. Participants were briefed verbally on the study purpose and data collected prior to installation, with formal written consent to be

obtained as part of the pending ethics review. Age was collected during in-app onboarding. Player IDs are UUIDs with no name linkage, consistent with PDPA principles (Singapore Personal Data Protection Act, 2012). No passive sensing data such as location, accelerometer, or microphone is collected.

Duty of care for cognitive decline signals. The professional consultation trigger is a sensitive feature. When sustained decline is detected, the system displays: *"We've noticed a steady decline over several sessions. Consider speaking with a healthcare professional."* The phrasing is deliberately encouraging rather than alarming, and the feature is framed as a wellness indicator rather than a diagnostic tool. This approach aligns with the emphasis on transparency and responsible framing in digital mental health tools, as advocated by Torous and Roberts (2017).

7.4 Future Work

Longer longitudinal study. A 3 to 6 month study with 60 to 100 participants stratified by age and MCI risk would provide the statistical power needed to detect meaningful changes in standardised cognitive assessment scores and would allow the cognitive trend detector to demonstrate its utility.

Contextual bandits with richer features. The on-device Thompson Sampling (ReminderAI.cs) operates over a 192-state space. The Cloud Function hour-selection bandit uses hour of day as the sole arm. Incorporating day-of-week, recent in-game activity level, and device unlock patterns into the CF bandit could further improve timing accuracy, analogous to the approach in HeartSteps (Klasnja et al., 2019).

iOS port. The application should be ported to iOS to reach Apple device users. Unity's cross-platform architecture means most game code can be reused. The primary changes would be in the notification and Firebase integration layers.

Multiplayer or social comparison features. Gerling et al. (2012) found that social comparison features such as leaderboards visible to family members or caregivers significantly increase older adult engagement with health games. A family dashboard that allows designated caregivers to view trend reports and session frequency could both increase accountability and provide clinically useful information to informal care networks.

Integration with wearable sensors. Passive data such as heart rate variability, sleep quality, and step count from smartwatches could be used as additional input features for the cognitive trend detector. Rawtaer et al. (2020) found that these sensor-derived metrics distinguished MCI patients from cognitively healthy individuals in a Singapore community sample, suggesting their value as complementary signals alongside cognitive assessments.

Deep RL for long-horizon difficulty adaptation. The current Q-Learning agent makes per-round difficulty decisions within sessions, with the Q-table persisted across sessions. A cross-session deep RL approach using a recurrent network over session history could model longer-term learning curves and better distinguish cognitive improvement from fatigue within a session.

Chapter 8: Conclusion

This report has presented CogniFarm, a mobile health application designed to support cognitive training and long-term engagement in older adults through two complementary AI-driven personalisation mechanisms: Q-Learning adaptive difficulty for in-game challenge calibration, and Thompson Sampling for personalised push notification timing to sustain out-of-game adherence.

The project addressed a genuine and significant real-world problem. Cognitive decline affects millions of older adults globally, and the window for effective intervention is narrow and often missed. mHealth applications have the potential to deliver cognitive training at scale, but long-term adherence remains the critical bottleneck. CogniFarm

takes two specific, technically grounded steps toward solving this problem: it keeps players in their optimal challenge zone by continuously learning their difficulty response curve, and it learns when to reach out to each individual user to achieve maximum engagement.

The technical contributions are summarised as follows. The adaptive difficulty system uses a 60-state tabular Q-Learning agent with informed prior initialisation and epsilon-greedy exploration, achieving sensible difficulty adjustment from the first session without requiring a cold-start exploration phase. The notification timing system implements a per-user Beta-Bernoulli Thompson Sampling bandit in a serverless Firebase Cloud Function, maintaining entirely separate posteriors per user and converging to each user's personal optimal engagement hour. The cognitive trend detector uses ordinary least squares linear regression over a configurable session window to monitor accuracy and response time trajectories, and raises a professional consultation flag when sustained decline over 8 sessions is detected.

Experiment 1 demonstrated, through simulation calibrated against the HeartSteps V1 and Oralytics datasets, that Thompson Sampling achieves 98.4% notification acceptance rate compared to 29.0% for the fixed rule-based baseline on HeartSteps V1, and leads on days active per 30-day episode (22.5 vs 21.2 days), with consistent ranking confirmed across both datasets.

Beyond the technical results, this project contributes a reproducible simulation framework for pre-deployment mHealth RL validation, a design pattern for per-user bandit state management in Firestore, and a Unity engineering reference for combining Firebase, Q-Learning, and mobile notifications in a single application.

The limitations of this work, primarily the short study duration and small sample, are inherent to the FYP format and do not diminish the validity of the proof-of-concept. The platform is ready for deployment in a longer follow-up study, and the architecture is designed to scale to a larger participant pool without changes to the backend.

Cognitive health is a deeply personal and often frightening topic for older adults and their families. CogniFarm's contribution is modest in isolation, but it demonstrates that the tools of modern machine learning can be embedded in consumer-grade mobile applications in ways that are both technically sound and genuinely useful to the people who need them. Personalisation, persistence, and the right nudge at the right moment: these are the engineering primitives from which effective cognitive health support can be built.

References

Agrawal, S. and Goyal, N. (2012). Analysis of Thompson Sampling for the multi-armed bandit problem. *Proceedings of the 25th Annual Conference on Learning Theory (COLT)*, PMLR 23, pp. 39.1-39.26.

Alzheimer's Disease International. (2023, September 21). *World Alzheimer Report 2023*. Alzheimer's Disease International (ADI). https://www.alzint.org/resource/world-alzheimer-report-2023/

Anguera, J. A., Boccanfuso, J., Rintoul, J. L., Al-Hashimi, O., Faraji, F., Janowich, J., Kong, E., Larraburo, Y., Rolle, C., Johnston, E., & Gazzaley, A. (2013). Video game training enhances cognitive control in older adults. *Nature*, *501*(7465), 97–101. https://doi.org/10.1038/nature12486

Barnard, Y., Bradley, M. D., Hodgson, F., & Lloyd, A. D. (2013). Learning to use new technologies by older adults: Perceived difficulties, experimentation behaviour and usability. *Computers in Human Behavior*, *29*(4), 1715–1724. https://doi.org/10.1016/j.chb.2013.02.006

Baumel, A., Muench, F., Edan, S., & Kane, J. M. (2019). Objective user engagement with mental health apps: Systematic search and panel-based usage analysis. *Journal of Medical Internet Research*, *21*(9). https://doi.org/10.2196/14567

Bidargaddi, N., Almirall, D., Murphy, S., Nahum-Shani, I., Kovalcik, M., Pituch, T., Maaieh, H., & Strecher, V. (2018). To prompt or not to prompt? A microrandomized trial of time-varying push notifications to increase proximal engagement with a mobile health app. *JMIR mHealth and uHealth*, *6*(11). https://doi.org/10.2196/10123

Bonnechère, B., Langley, C., & Sahakian, B. J. (2020). The use of commercial computerised cognitive games in older adults: A meta-analysis. *Scientific Reports*, *10*(1). https://doi.org/10.1038/s41598-020-72281-3

Brühlmann, Florian. (2013). Gamification From the Perspective of Self-Determination Theory and Flow. 10.13140/RG.2.1.1181.8080.

Burke, J. W., McNeill, M. D., Charles, D. K., Morrow, P. J., Crosbie, J. H., & McDonough, S. M. (2009). Optimising engagement for stroke rehabilitation using serious games. *The Visual Computer*, *25*, 1085–1099. https://doi.org/10.1007/s00371-009-0387-4

Cafazzo, J. A., Casselman, M., Hamming, N., Katzman, D. K., & Palmert, M. R. (2012). Design of an mHealth app for the self-management of adolescent type 1 diabetes: A pilot study. *Journal of Medical Internet Research*, *14*(3). https://doi.org/10.2196/jmir.2058

Chapelle, O. and Li, L. (2011). An empirical evaluation of Thompson Sampling. *NIPS'11: Proceedings of the 25th International Conference on Neural Information Processing Systems* pp. 2249-2257. https://dl.acm.org/doi/10.5555/2986459.2986710

Chen, R., Young, K., Chao, L. L., Miller, B., Yaffe, K., Weiner, M. W., & Herskovits, E. H. (2012). Prediction of conversion from mild cognitive impairment to alzheimer disease based on Bayesian Data Mining with Ensemble learning. *The Neuroradiology Journal*, *25*(1), 5–16. https://doi.org/10.1177/197140091202500101

Corsi, P. M. (1972). *Human memory and the medial temporal region of the brain.* Doctoral dissertation, McGill University.

Csikszentmihalyi, M. (1990). *Flow: The Psychology of Optimal Experience.* Harper and Row, New York.

Daniel, J. R., Benjamin, V. Roy., Abbas, Kazerouni., Ian, Osband., & Zheng, Wen. (2018). A tutorial on Thompson sampling. *Foundations and Trends® in Machine Learning*, *11*(1), 1–99. https://doi.org/10.1561/2200000070

Dayer, L., Heldenbrand, S., Anderson, P., Gubbins, P. O., & Martin, B. C. (2013). Smartphone medication adherence apps: Potential benefits to patients and providers. *Journal of the American Pharmacists Association*, *53*(2), 172–181. https://doi.org/10.1331/japha.2013.12202

Deci, E. L. and Ryan, R. M. (1985). *Intrinsic Motivation and Self-Determination in Human Behavior.* Plenum, New York.

Deterding, S., Dixon, D., Khaled, R. and Nacke, L. (2011). From game design elements to gamefulness: Defining gamification. *Proceedings of MindTrek '11*, ACM, pp. 9-15.

de Jager Loots, C. A., Price, G., Barbera, M., Neely, A. S., Gavelin, H. M., Lehtisalo, J., Ngandu, T., Solomon, A., Mangialasche, F., & Kivipelto, M. (2024). Development of a cognitive training support programme for prevention of dementia and cognitive decline in at-risk older adults. *Frontiers in Dementia*, *3*. https://doi.org/10.3389/frdem.2024.1331741

Deterding, S., Dixon, D., Khaled, R., & Nacke, L. (2011). From game design elements to gamefulness. *Proceedings of the 15th International Academic MindTrek Conference: Envisioning Future Media Environments*, 9–15. https://doi.org/10.1145/2181037.2181040

Falzarano, F. (2023). Technology and older adults: Best practices for enhancing usability, engagement, and Communication. *Innovation in Aging*, *7*(Supplement_1), 350–351. https://doi.org/10.1093/geroni/igad104.1167

Gerling, K. M., Schulte, F. P., Smeddinck, J., & Masuch, M. (2012). Game design for older adults: Effects of age-related changes on structural elements of Digital Games. *Lecture Notes in Computer Science*, 235–242. https://doi.org/10.1007/978-3-642-33542-6_20

Hardy, J. L., Nelson, R. A., Thomason, M. E., Sternberg, D. A., Katovich, K., Farzin, F., & Scanlon, M. (2015). Enhancing cognitive abilities with comprehensive training: A large, online, randomized, active-controlled trial. *PLOS ONE*, *10*(9). https://doi.org/10.1371/journal.pone.0134467

Hunicke, R. (2005). The case for Dynamic Difficulty Adjustment in games. *Proceedings of the ACM Workshop on Adaptive and Intelligent Game Interfaces*, pp. 429-433.

Kakulla, B. (2023). *2024 Tech Trends and Adults 50-Plus*. https://doi.org/https://doi.org/10.26419/res.00772.001

Kaufmann, E., Korda, N., & Munos, R. (2012). Thompson sampling: An asymptotically optimal finite-time analysis. *Lecture Notes in Computer Science*, 199–213. https://doi.org/10.1007/978-3-642-34106-9_18

Klasnja, P., Smith, S., Seewald, N. J., Lee, A., Hall, K., Luers, B., Hekler, E. B., & Murphy, S. A. (2019). Efficacy of contextually tailored suggestions for physical activity: A micro-randomized optimization trial of heartsteps. *Annals of Behavioral Medicine*, *53*(6), 573–582. https://doi.org/10.1093/abm/kay067

Kueider, A. M., Parisi, J. M., Gross, A. L., & Rebok, G. W. (2012). Computerized cognitive training with older adults: A systematic review. *PLoS ONE*, *7*(7). https://doi.org/10.1371/journal.pone.0040588

Lai, T. L., & Robbins, H. (1985). Asymptotically efficient adaptive allocation rules. *Advances in Applied Mathematics*, *6*(1), 4–22. https://doi.org/10.1016/0196-8858(85)90002-8

Lampit, A., Hallock, H., & Valenzuela, M. (2014). Computerized cognitive training in cognitively healthy older adults: A systematic review and meta-analysis of effect modifiers. *PLoS Medicine*. https://doi.org/10.1371/journal.pmed.1001756

Liao, P., Klasnja, P., & Murphy, S. (2020). Off-policy estimation of long-term average outcomes with applications to Mobile Health. *Journal of the American Statistical Association*, *116*(533), 382–391. https://doi.org/10.1080/01621459.2020.1807993

Lopes, R., & Bidarra, R. (2011). Adaptivity challenges in games and simulations: A survey. *IEEE Transactions on Computational Intelligence and AI in Games*, *3*(2), 85–99. https://doi.org/10.1109/tciaig.2011.2152841

McGoldrick, C., Crawford, S., & Evans, J. J. (2019). MindMate: A single case experimental design study of a reminder system for people with dementia.

Neuropsychological Rehabilitation, *31*(1), 18–38. https://doi.org/10.1080/09602011.2019.1653936

Miralles, I., Granell, C., Díaz-Sanahuja, L., Van Woensel, W., Bretón-López, J., Mira, A., Castilla, D., & Casteleyn, S. (2020). Smartphone apps for the treatment of Mental Disorders: Systematic Review. *JMIR mHealth and uHealth*, *8*(4). https://doi.org/10.2196/14897

National Institute for Health and Care Excellence. (2022). *Evidence standards framework for digital health technologies*. NICE. https://www.nice.org.uk/guidance/ecd7

Ng, M. M., Firth, J., Minen, M., & Torous, J. (2019). User engagement in Mental Health Apps: A Review of Measurement, reporting, and validity. *Psychiatric Services*, *70*(7), 538–544. https://doi.org/10.1176/appi.ps.201800519

Ngandu, T., Lehtisalo, J., Solomon, A., Levälahti, E., Ahtiluoto, S., Antikainen, R., Bäckman, L., Hänninen, T., Jula, A., Laatikainen, T., Lindström, J., Mangialasche, F., Paajanen, T., Pajala, S., Peltonen, M., Rauramaa, R., Stigsdotter-Neely, A., Strandberg, T., Tuomilehto, J., … Kivipelto, M. (2015). A 2 year multidomain intervention of diet, exercise, cognitive training, and vascular risk monitoring versus control to prevent cognitive decline in at-risk elderly people (finger): A randomised controlled trial. *The Lancet*, *385*(9984), 2255–2263. https://doi.org/10.1016/s0140-6736(15)60461-5

Petersen, R. C. (2004). Mild cognitive impairment as a diagnostic entity. *Journal of Internal Medicine*, *256*(3), 183–194. https://doi.org/10.1111/j.1365-2796.2004.01388.x

Petersen, Ronald C., & Negash, S. (2008). Mild cognitive impairment:*an overview*. *CNS Spectrums*, *13*(1), 45–53. https://doi.org/10.1017/s1092852900016151

Petersen, Ronald C., Smith, G. E., Waring, S. C., Ivnik, R. J., Tangalos, E. G., & Kokmen, E. (1999). Mild cognitive impairment. *Archives of Neurology*, *56*(3), 303–308. https://doi.org/10.1001/archneur.56.3.303

Rabbi, M., Aung, M. H., Zhang, M., & Choudhury, T. (2015). MyBehavior. *Proceedings of the 2015 ACM International Joint Conference on Pervasive and Ubiquitous Computing*, 707–718. https://doi.org/10.1145/2750858.2805840

Rawtaer, I., Mahendran, R., Kua, E. H., Tan, H. P., Tan, H. X., Lee, T.-S., & Ng, T. P. (2020). Early detection of mild cognitive impairment with in-home sensors to monitor behavior patterns in community-dwelling senior citizens in Singapore: Cross-sectional Feasibility Study. *Journal of Medical Internet Research*, *22*(5). https://doi.org/10.2196/16854

Robbins, T. W., James, M., Owen, A. M., Sahakian, B. J., McInnes, L., & Rabbitt, P. (1994). Cambridge Neuropsychological Test Automated Battery (cantab): A Factor Analytic Study of a large sample of normal elderly volunteers. *Dementia and Geriatric Cognitive Disorders*, *5*(5), 266–281. https://doi.org/10.1159/000106735

Roenneberg, T., Kuehnle, T., Juda, M., Kantermann, T., Allebrandt, K., Gordijn, M., & Merrow, M. (2007). Epidemiology of the human circadian clock. *Sleep Medicine Reviews*, *11*(6), 429–438. https://doi.org/10.1016/j.smrv.2007.07.005

Russo, D., Van Roy, B., Kazerouni, A. and Osband, I. (2018). A tutorial on Thompson Sampling. *Foundations and Trends in Machine Learning*, 11(1), pp. 1-96. https://doi.org/10.48550/arXiv.1707.02038

Salthouse, T. A. (2000). Aging and measures of processing speed. *Biological Psychology*, *54*(1–3), 35–54. https://doi.org/10.1016/s0301-0511(00)00052-1

Salthouse, T. A. (2019). Trajectories of normal cognitive aging. *Psychology and Aging*, *34*(1), 17–24. https://doi.org/10.1037/pag0000288

Shatil, E. (2013). Does combined cognitive training and physical activity training enhance cognitive abilities more than either alone? A four-condition randomized controlled trial among healthy older adults. *Frontiers in Aging Neuroscience*, *5*(8). https://doi.org/10.3389/fnagi.2013.00008

Simons, D. J., Boot, W. R., Charness, N., Gathercole, S. E., Chabris, C. F., Hambrick, D. Z., & Stine-Morrow, E. A. (2016). Do “brain-training” programs work? *Psychological Science in the Public Interest*, *17*(3), 103–186. https://doi.org/10.1177/1529100616661983

Singapore Statutes Online. (2012). *Personal Data Protection Act 2012*. https://sso.agc.gov.sg/Act/PDPA2012

Smith, G. E., Housen, P., Yaffe, K., Ruff, R., Kennison, R. F., Mahncke, H. W., & Zelinski, E. M. (2009). A cognitive training program based on principles of brain plasticity: Results from the improvement in memory with plasticity-based Adaptive Cognitive training (IMPACT) study. *Journal of the American Geriatrics Society*, *57*(4), 594–603. https://doi.org/10.1111/j.1532-5415.2008.02167.x

Stern, Y. (2012). Cognitive Reserve in ageing and alzheimer’s disease. *The Lancet Neurology*, *11*(11), 1006–1012. https://doi.org/10.1016/s1474-4422(12)70191-6

Sutton, R. S. and Barto, A. G. (2018). *Reinforcement Learning: An Introduction.* 2nd edition. MIT Press, Cambridge, MA.

Tewari, A., Murphy, S.A. (2017). From Ads to Interventions: Contextual Bandits in Mobile Health. In: Rehg, J., Murphy, S., Kumar, S. (eds) Mobile Health. Springer, Cham. https://doi.org/10.1007/978-3-319-51394-2_25

Thaler, R. H., Sunstein, C. R. and Balz, J. P. (2013). Choice architecture. In: *The Behavioral Foundations of Public Policy* (ed. Shafir, E.), Princeton University Press, pp. 428-439. https://doi.org/10.1515/9781400845347-029

Thompson, W. R. (1933). On the likelihood that one unknown probability exceeds another in view of the evidence of two samples. *Biometrika*, 25(3-4), pp. 285-294.

Trella, A. L., Zhang, K. W., Carpenter, S. M., Elashoff, D., Greer, Z. M., Nahum-Shani, I., Ruenger, D., Shetty, V. and Murphy, S. A. (2024) 'Oralytics Reinforcement Learning Algorithm', *arXiv preprint*, arXiv:2406.13127.

Torous, J., & Roberts, L. W. (2017). Needed innovation in digital health and smartphone applications for Mental Health. *JAMA Psychiatry*, *74*(5), 437. https://doi.org/10.1001/jamapsychiatry.2017.0262

Trafton, J. G., Altmann, E. M., Brock, D. P., & Mintz, F. E. (2003). Preparing to resume an interrupted task: Effects of prospective goal encoding and retrospective rehearsal. *International Journal of Human-Computer Studies*, *58*(5), 583–603. https://doi.org/10.1016/s1071-5819(03)00023-5

Vygotsky, L. S. (1978). *Mind in Society: The Development of Higher Psychological Processes.* Harvard University Press, Cambridge, MA.

Watkins, C. J., & Dayan, P. (1992). Q-learning. *Machine Learning*, *8*(3–4), 279–292. https://doi.org/10.1007/bf00992698

World Health Organization (2022). *Ageing and Health.* WHO Fact Sheet. Available at: https://www.who.int/news-room/fact-sheets/detail/ageing-and-health [Accessed March 2026].

World Health Organization (2023). *Dementia.* WHO Fact Sheet. Available at: https://www.who.int/news-room/fact-sheets/detail/dementia [Accessed March 2026].

World Health Organization. (2025a, March 31). *Dementia*. https://www.who.int/en/news-room/fact-sheets/detail/dementia

World Health Organization. (2025b, October 1). *Ageing and health*. https://www.who.int/news-room/fact-sheets/detail/ageing-and-health

Wu, J. (2024). In-depth exploration and implementation of multi-armed bandit models across diverse fields. *Highlights in Science, Engineering and Technology*, *94*, 201–205. https://doi.org/10.54097/d3ez0n61

Appendices

Appendix A: Q-Table Initialisation Pseudocode

```
// Q-Table: float[4, 3, 5, 3]
// Dimensions: [performance_level, response_time_level, difficulty_index, action]
// Actions: 0=Decrease, 1=Maintain, 2=Increase

for p in 0..3:         // performance levels
    for r in 0..2:     // response time levels
        for d in 0..4: // difficulty levels (0-indexed)

            if p == 0:  // Poor performance (<40% accuracy)
                Q[p,r,d,0] = +1.0   // strongly decrease
                Q[p,r,d,1] = -0.5   // penalise maintain
                Q[p,r,d,2] = -1.0   // strongly penalise increase

            elif p == 1:  // Fair performance (40-60%)
                Q[p,r,d,0] = +0.3   // slight decrease preferred
                Q[p,r,d,1] = +0.5   // maintain acceptable
                Q[p,r,d,2] = -0.3   // slight penalty for increase

            elif p == 2:  // Good performance (60-80%) - TARGET ZONE
                Q[p,r,d,0] = -0.3   // penalise decrease
                Q[p,r,d,1] = +1.0   // strongly maintain
                Q[p,r,d,2] = +0.3   // slight increase reward

            else:  // p == 3: Excellent performance (>80%)
                Q[p,r,d,0] = -1.0   // strongly penalise decrease
                Q[p,r,d,1] = -0.3   // penalise maintain
                Q[p,r,d,2] = +1.0   // strongly increase

            // Add small random noise to break symmetry
            for a in 0..2:
                Q[p,r,d,a] += Uniform(-0.05, 0.05)
```

Appendix B: Thompson Sampling Cloud Function (functions/index.js)

```
const { onSchedule } = require("firebase-functions/v2/scheduler");
const { onRequest } = require("firebase-functions/v2/https");
const admin = require("firebase-admin");

admin.initializeApp();
const db = admin.firestore();

const DEFAULT_REMINDER_HOUR = 10;

// ── Beta sampling via Box-Muller normal approximation ──────────────────
// Accurate when α+β > 5; clipped to (0.01, 0.99).
function sampleBeta(alpha, beta) {
  const mu  = alpha / (alpha + beta);
  const v   = (alpha * beta) / (Math.pow(alpha + beta, 2) * (alpha + beta + 1));
  const std = Math.sqrt(v);
  const u1  = Math.max(1e-10, Math.random());
  const u2  = Math.random();
  const z   = Math.sqrt(-2 * Math.log(u1)) * Math.cos(2 * Math.PI * u2);
  return Math.max(0.01, Math.min(0.99, mu + std * z));
}

// ── Warm-start prior: preferred hour gets α=3, all others α=1 ──────────
function initHourAlpha(preferredHour) {
  const alpha = Array(24).fill(1);
  const h = Number(preferredHour);
  if (h >= 0 && h < 24) alpha[h] = 3;
  return alpha;
}
```

```
// ── Thompson Sampling over 24 hour arms ──────────────────────────────────────
function tsSelectHour(alpha, beta) {
  const samples = alpha.map((a, h) => sampleBeta(a, beta[h]));
  return samples.indexOf(Math.max(...samples));
}

// ── Scheduled trigger: every hour ────────────────────────────────────────────
exports.sendDailyReminders = onSchedule(
  { schedule: "0 * * * *", timeZone: "Asia/Singapore", region: "asia-southeast1" },
  async () => {
    const currentHour = (new Date().getUTCHours() + 8) % 24;
    await sendRemindersForHour(currentHour);
    await evaluateProximalEngagement();
  }
);

// ── HTTP trigger: test mode (ignores hour filter) ────────────────────────────
exports.sendRemindersNow = onRequest(
  { region: "asia-southeast1" },
  async (req, res) => {
    const results = await sendRemindersForHour(null);
    res.json({ success: true, results });
  }
);

// ── Core send logic ──────────────────────────────────────────────────────────
async function sendRemindersForHour(hourFilter) {
  const playersSnap = await db.collection("players").get();
  const results = [];
  const todayStr = new Date().toISOString().split("T")[0];
```

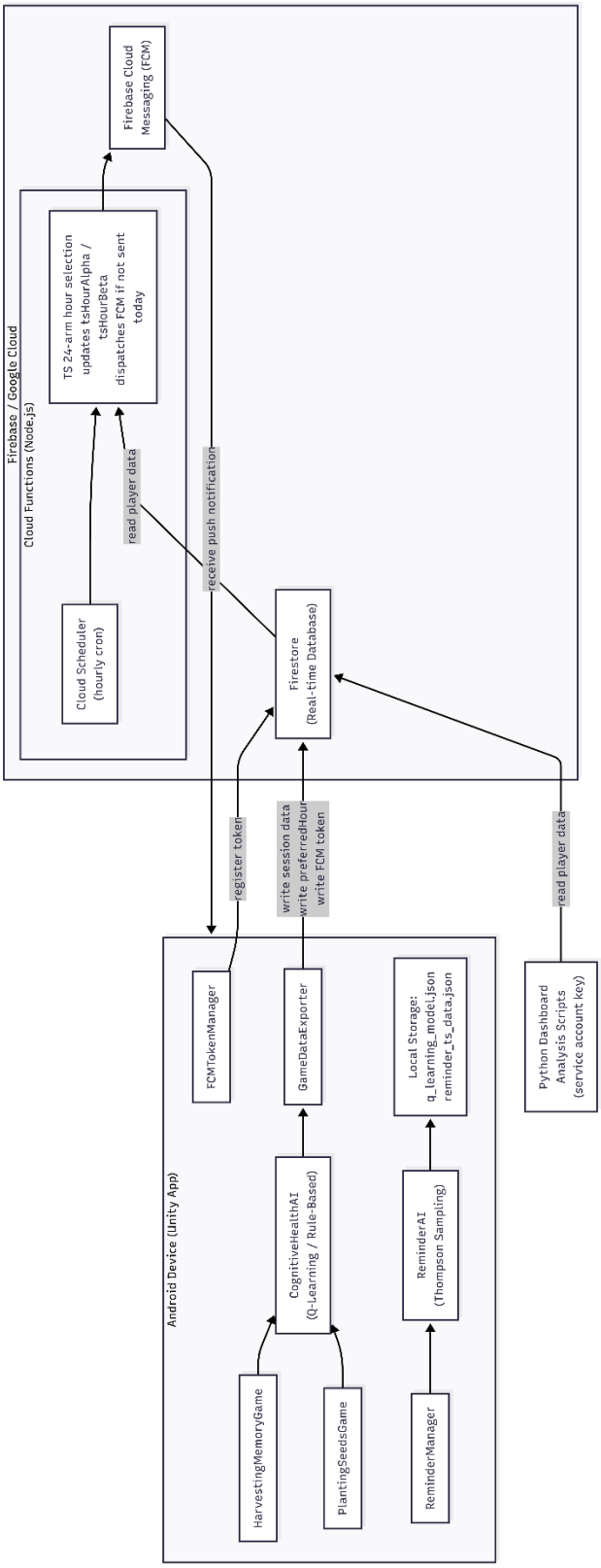
```
  for (const playerDoc of playersSnap.docs) {
    const data = playerDoc.data();
    const playerId = playerDoc.id;
    const isRLByStudyGroup = data.studyGroup === "rl";
    const aiMode = data.reminderAIMode ||
                   (isRLByStudyGroup ? "ThompsonSampling" : "RuleBased");
    const isRLGroup = aiMode === "ThompsonSampling" ||
                      aiMode === "QLearning" || isRLByStudyGroup;

    let playerHour;
    if (isRLGroup) {
      const alpha = data.tsHourAlpha ||
                    initHourAlpha(data.preferredHour ?? DEFAULT_REMINDER_HOUR);
      const beta  = data.tsHourBeta  || Array(24).fill(1);
      if (!data.tsHourAlpha) await playerDoc.ref.update({ tsHourAlpha: alpha,
tsHourBeta: beta });
      playerHour = tsSelectHour(alpha, beta);
    } else {
      playerHour = DEFAULT_REMINDER_HOUR;
    }

    if (hourFilter !== null && playerHour !== hourFilter) continue;
    if (data.lastReminderDate === todayStr) continue;
    if (!data.fcmToken) continue;

    const ruleDecision = getReminderDecision(data);
    if (!ruleDecision.shouldSend) continue;

    const useTSContent = isRLGroup && data.tsTitle && data.tsBody;
```

```
    const title = useTSContent ? data.tsTitle : ruleDecision.title;
    const body  = useTSContent ? data.tsBody  : ruleDecision.body;

    await admin.messaging().send({
      token: data.fcmToken,
      notification: { title, body },
      android: { priority: "high",
                 notification: { channelId: "cognifarm_reminders",
                                 color: "#4CAF50", sound: "default" } },
    });

    await playerDoc.ref.update({
      lastReminderDate: todayStr,
      lastNotificationSentAt: admin.firestore.FieldValue.serverTimestamp(),
      tsLastSentHour: playerHour,
    });

    results.push({ playerId, status: "sent", hour: playerHour });
  }
  return results;
}

// Proximal engagement evaluation (2-hour window) ──────────────────────────
async function evaluateProximalEngagement() {
  const TWO_HOURS_MS = 2 * 60 * 60 * 1000;
  const now = new Date();
  const playersSnap = await db.collection("players").get();

  for (const playerDoc of playersSnap.docs) {
    const data = playerDoc.data();
```

```
    if (!data.lastNotificationSentAt) continue;

    const sentAt = data.lastNotificationSentAt.toDate();

    if (now - sentAt < TWO_HOURS_MS) continue;

    if (data.lastNotificationEvaluated?.toDate().getTime() === sentAt.getTime())
continue;


    const lastSession = data.lastUpdated?.toDate() ?? null;

    const hoursDiff = lastSession ? (lastSession - sentAt) / 3600000 : null;

    const engaged = hoursDiff !== null && hoursDiff >= 0 && hoursDiff <= 2.0;


    // Update TS hour posteriors

    const isRLPlayer = data.studyGroup === "rl" ||

                       data.reminderAIMode === "ThompsonSampling";

    const sentHour = data.tsLastSentHour;

    if (isRLPlayer && sentHour != null) {

      const alpha = [...(data.tsHourAlpha || Array(24).fill(1))];

      const beta  = [...(data.tsHourBeta  || Array(24).fill(1))];

      if (engaged) alpha[sentHour] += 1; else beta[sentHour] += 1;

      await playerDoc.ref.update({ tsHourAlpha: alpha, tsHourBeta: beta });

    }


    // Write engagement signal to reminders subcollection

    const remSnap = await playerDoc.ref.collection("reminders")

                          .orderBy("sentAt", "desc").limit(1).get();

    if (!remSnap.empty) {

      await remSnap.docs[0].ref.update({

        engagementSignal: engaged ? "Accepted" : "Ignored",

        hoursBetweenNotifAndSession: hoursDiff ?? -1,

      });

    }
```

```
    await playerDoc.ref.update({ lastNotificationEvaluated:
data.lastNotificationSentAt });

  }

}
```

Appendix C: Enlarged Figure 1 - System Architecture Diagram



Appendix D: Enlarged Figure 8 - Cloud Function Scheduling Flowchart